

An Internet of AI for Cyber Security:

A Decentralised Multi-Agent Defence Simulation

BSc Computer Science & Artificial Intelligence
Loughborough University

Author: Louis Poole
Supervisor: Dr Andrea Soltoggio
Academic Year: 2025–2026

Abstract

This project investigates whether artificial intelligence (AI) agents, each independently defending a single machine on a network, can collectively provide effective cyber defence and whether they can match or exceed the protection offered by a traditional, centrally managed security system.

Using a network attack simulation framework developed by Microsoft Research, six defensive configurations were tested across 2,500 simulated attack episodes each: no defence, a centralised defender, three decentralised configurations with varying levels of information sharing between agents, and a self-learning AI defender trained using reinforcement learning.

Four research questions were investigated: whether decentralised agents can provide effective defence (RQ1), whether cooperation improves outcomes (RQ2), how decentralised compares to centralised (RQ3), and whether Reinforcement Learning can outperform rule-based approaches (RQ4).

All four research questions were answered with statistically significant results:

- Every defensive configuration reduced network compromise compared to no defence.
- Decentralised agents operating with only local information matched the performance of a centralised defender. Thus, providing equivalent protection without the infrastructure overhead of a central system.
- Restricting alert sharing to direct neighbours proved no less effective than broadcasting alerts across the entire network.
- A trained reinforcement learning agent outperformed all rule-based defenders, reducing both the spread of attacks and the value extracted by attackers.

In addition:

- Reward hacking, a known failure mode of self-learning AI, was encountered and resolved during training, providing a concrete case study in responsible AI deployment for security applications.

Acknowledgements

I would firstly like to thank my parents and my significant other for their unconditional support and countless proof-reads. Without them I would not have had the willpower and passion to complete this thesis in the way I envisioned it.

I would also like to thank my supervisor Mr. Soltoggio for helping me shape the direction of my thesis and for his invaluable guidance in the latter stages of writing. Furthermore, for being a genuinely engaging and approachable lecturer and educator.

A further thank you is given to Christian Farrell (Security Strategy Lead @ Accenture Security) for his invaluable feedback and industry insider perspective on the current challenges and solutions being faced and proposed in industry.

I would finally also like to extend a big thank you to my host company in Australia, not only for bringing me onboard but for teaching me the ins and outs of being a part of a mature cyber security team. My learnings both in and out of the office were a big motivator in choosing this thesis topic and direction.

Table of Contents

Contents

Abstract.....	2
Acknowledgements.....	3
Table of Contents.....	4
1. Introduction	7
1.1 Motivation	7
1.2 The Changing Threat Landscape	8
1.3 Research Questions	9
1.4 Contributions	9
1.5 Report Structure	10
2. Background and Literature Review	11
2.1 Background	11
2.1.1 How Cyber Attacks Propagate	11
2.1.2 Reinforcement Learning	11
2.1.3 Cyber Attack Simulation	12
2.1.4 Multi-Agent Systems	13
2.2 Literature Review.....	13
2.2.1 The Changing SOC Paradigm.....	13
2.2.2 The Case for AI-Driven Defence: Policy Endorsements and Threat Escalation	15
2.2.3 Deep Q-Networks (DQN)	16
2.2.4 Reinforcement Learning Meets Cyber Defence	17
2.2.5 Prior Quantitative Results and Benchmarks	19
2.2.6 Gaps in Literature.....	20
3. Simulation Environment & Experimental Design	21
3.1 The CyberBattleSim Framework.....	21
3.1.1 Framework Architecture	21
3.1.2 Attacker Model	22
3.2 Network Topology.....	22
3.3 Experimental Conditions.....	23
3.4 Detection Model: Grounding in Real-World Data	23
3.5 Evaluation Metrics	24
3.6 Statistical Analysis.....	24
3.7 Implementation Challenges and Solutions	25
Challenge 1: Node ID Type Mismatch (Silent NOOP Failure).....	25
Challenge 2: NumPy 2.x Incompatibility	25

Challenge 3: Agent Reinitialisation Across Episodes (Epsilon Stagnation).....	25
Challenge 4: Time-to-First-Compromise Metric Always Recording Zero.....	25
Challenge 5: Compute Platform — Migration from Google Colab to Kaggle.....	26
Challenge 6: Reward Hacking — DQN Learning an Always-Reimage Policy	26
4. Rule-Based Defender Architecture	27
4.1 System Design	27
4.2 Suspicion Accumulation.....	27
4.3 Decision Policy	27
4.4 Alert Sharing Models	28
4.5 Fairness Constraints.....	28
5. Reinforcement Learning Defender	29
5.1 Architecture Overview.....	29
5.2 What Each Agent Observes.....	29
5.3 Neural Network Architecture	29
5.4 Reward Function	30
5.5 Training	31
5.6 Hyperparameters.....	32
6. Results.....	33
6.1 Summary of All Conditions	33
6.2 Statistical Significance	35
6.3 Finding 1 — RQ1: All Defenders Significantly Reduce Compromise.....	36
6.4 Finding 2 — RQ3: Decentralised Agents Match Centralised Performance.....	37
6.5 Finding 3 — RQ2: Cooperative vs Competitive Alert Sharing	37
6.6 Finding 4 — RQ4: DQN Outperforms All Heuristics	38
6.7 Reward Hacking: A Training Failure and Its Resolution	38
7. Discussion	40
7.1 Decentralised Defence as a Viable Architecture	40
7.2 Implications for Threat Intelligence Sharing Policy (ISAC)	40
7.3 Implications for Organisations.....	41
Understanding the cost of centralised security	41
The reward hacking lesson for AI security procurement.....	41
7.4 Limitations	42
8. Conclusion	43
References	44
Appendix A: Project File Structure.....	48
Appendix B: Development Environment.....	49

Appendix C: Original vs. Final Project Plan	50
Project Evolution	50
Project Timeline.....	52

1. Introduction

1.1 Motivation

My main motivation for choosing to investigate how AI could be used as a viable strategy in cyber defence came after my degree's year in industry. During that year, I was stationed in both Melbourne and Sydney within one of Australia's largest alternative financial asset managers as a Junior Security Analyst. My responsibilities included: email legitimacy verification, deep-web reconnaissance, threat escalation, alert, and endpoint monitoring. I was also able to lead some projects of my own, including the adoption of AI tools within our processes to speed up and improve the accuracy of our security team.

Upon arrival, one of the first things I learnt was that the financial services sector is a particularly demanding environment for cyber defence. Organisations of this kind are high-value targets given they hold sensitive client financial data and execute large-value transactions daily; all of this while operating under strict regulatory obligations. I learned that in Australia, the APRA's CPS 234 mandates robust information security capabilities and clear accountability for cyber resilience. This meant that any proposal to introduce new tooling, including AI, carried compliance overhead (known as Regulatory Risk-management) from the outset; making the barrier to adoption higher than it might be in less regulated industries.

While the years' experience was not only incredibly formative, it also was insightful as it offered me an unfiltered look into the real "boots-on-the-ground" issues current cyber defence teams are facing now (and will increasingly face in the not-so-distant future). Many of the simpler tasks rapidly became mundane and very repetitive, despite their importance in the company's overall security posture. The volume of daily alerts meant that triage queues would consistently back up, and the risk of genuine threats being missed amid the noise was a real concern. This is by no means a criticism of the team; it is a structural consequence of asking skilled analysts to perform high-volume, low-complexity work at an ever-increasing pace.

When it was suggested that Artificial Intelligence (AI) tools could not only assist but maybe replace certain roles or tasks within the team, the responses were always the same: "We're not sure the available AI tools are mature and secure enough" or "Adding AI tools will just create more work and overhead on our end". What made these answers difficult to accept was that they coexisted with an observable reality: AI tools were already being trialled and adopted in pockets of the team's work, including in projects I was leading.

The question the team was really asking was not "can AI do this?" but "can we trust AI to do this without a human in the loop, and what evidence would we need to believe that?" Many members of the team, myself included, felt that we could contribute more meaningfully if freed from high-volume, repetitive monitoring tasks. That sentiment was shared not only among junior analysts but at senior and managerial levels too. This suggested the problem was not one of individual capability but of resource allocation and perceived risk.

If the results of this thesis show that decentralised local agents can match centralised performance, that restricted information sharing is defensively sufficient, and that a trained reinforcement learning agent substantially outperforms rule-based heuristics, then organisations like the one I worked in have a concrete, evidence-based reason to reconsider their position. Not as a replacement for skilled analysts, but as an always-on first line of response that handles

the repetitive, high-volume work autonomously. In turn freeing human expertise for the judgement calls that genuinely require it.

1.2 The Changing Threat Landscape

Cyber-attacks are no longer the exclusive domain of highly skilled, well-resourced adversaries. The widespread availability of AI-powered tools has made it possible for attackers with limited technical expertise and tools to launch sophisticated intrusions at scale. The same technological shift that has made AI a productive tool in business and research has also lowered the barrier to launching complex, coordinated cyber-attacks.

The global average attacker dwell time (the number of days between an attacker first entering a network and being detected) stood at 11 days in 2025 (Mandiant M-Trends, 2025). This figure, while improved from earlier years, still represents 11 days during which an attacker can move freely through a network: accessing sensitive data, escalating their level of control, and preparing for a more damaging decisive action.

The most usual form of organised cyber defence is the Security Operations Centre (SOC): a team of analysts who monitor network activity from a central location, investigate alerts, and coordinate responses. It is important to note at this stage that SOC's do not operate alone (security controls operate in parallel) but act as the coordination layer. Therefore, its structural limitations propagate to the entire stack.

While effective, centralised SOC's face many structural limitations. First, they represent a single point of failure; if the SOC is compromised or overwhelmed, the entire network loses coordinated defence. Second, they scale poorly; the cost and complexity of monitoring thousands of machines from a single vantage point grows rapidly. Third, they introduce a bottleneck; every decision passes through a central system, adding latency to detection and response.

Typical Security Operations Centre (SOC) Structure

High alert volumes flow upward through tiers, with escalation decisions at each level. All coordination passes through a central platform.

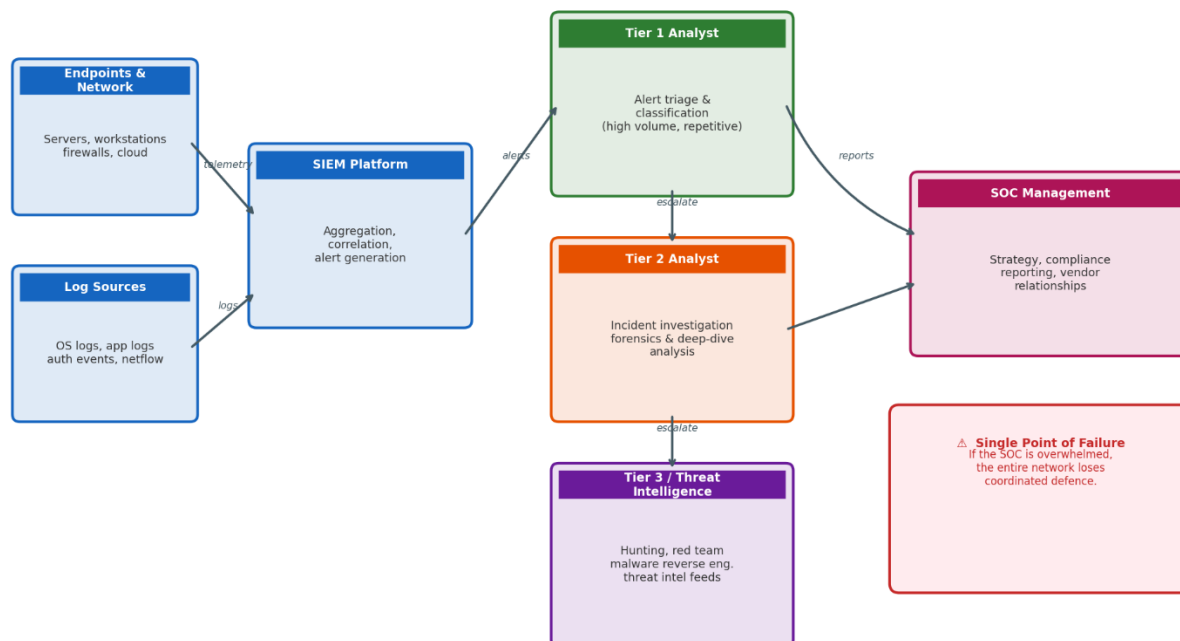


Figure 1. Typical Security Operations Centre (SOC) structure. Alerts generated by network endpoints flow through a central SIEM platform to tiered analyst teams, with escalation decisions at each level. All coordination passes through a centralised platform, creating both a bottleneck and a single point of failure.

This project investigates whether a fundamentally different approach, deploying small, independent AI agents at each machine rather than a single central system, can provide equivalent or better protection while addressing these structural weaknesses.

The personal motivation behind this investigation, drawn from industry experience as a security analyst, is described in Section 1.1

1.3 Research Questions

Four specific questions are investigated:

RQ	Question	Outcome
RQ1	Can decentralised AI agents provide effective cyber defence in simulated networks?	Yes - all defenders significantly reduced compromise ($p < 0.001$)
RQ2	Does cooperation between defenders improve detection and containment?	Yes, but cooperation intensity yields no significant additional benefit (cooperative vs. competitive: $p = 0.166$, $d = 0.040$).
RQ3	How does decentralised defence compare with centralised?	Statistically equivalent performance ($p > 0.05$)
RQ4	Can reinforcement learning improve defensive decision policies beyond rule-based approaches?	Yes - DQN outperforms all heuristics ($p < 0.001$, Cohen's $d = 0.509$)

Table 1. Research questions and high-level outcomes.

1.4 Contributions

The primary contributions of this project are:

- A working multi-agent cyber defence simulation supporting six experimental conditions across 5 independent runs of 500 episodes each.
- A novel experimental distinction between cooperative (full network broadcast) and competitive (neighbour-only) alert sharing, mapped to real-world intelligence sharing frameworks.
- Empirical evidence that decentralised local-only agents match centralised defender performance (i.e. SOC performance) at statistically equivalent levels.
- Empirical evidence that restricted alert sharing achieves equivalent outcomes to full broadcast, directly relevant to privacy-preserving threat intelligence policy.
- A trained reinforcement learning defender that outperforms all rule-based conditions, with a documented case study of reward hacking and its resolution.

1.5 Report Structure

Chapter 2 provides background on the technical concepts underpinning the project and reviews the relevant academic literature, identifying the gaps this work addresses.

Chapter 3 describes the simulation environment and experimental design.

Chapter 4 details the rule-based defender architecture.

Chapter 5 covers the reinforcement learning defender architecture.

Chapter 6 presents experimental results.

Chapter 7 discusses findings and their implications for real organisations.

Chapter 8 concludes with limitations and future directions.

2. Background and Literature Review

2.1 Background

2.1.1 How Cyber Attacks Propagate

Understanding the problem this project addresses requires a brief description of how attackers move through networks. The Cyber Kill Chain, developed by Hutchins et al. (2011) at Lockheed Martin, describes a cyber intrusion as a sequence of stages: initial reconnaissance, delivering a payload, gaining a foothold, establishing remote control (Command and Control, or C2), and finally taking action (stealing data, encrypting files, or disrupting services).

This project focuses specifically on the final two stages: the attacker already has a means of attack for one machine and can move laterally through the network to maximise its reach. This lateral movement phase is where defenders have the greatest opportunity to intervene, and where the architectural choice between centralised and decentralised defence has the most practical impact.

The MITRE ATT&CK framework (MITRE Corporation, 2022) provides a detailed breakdown of the specific techniques attackers use during this phase, such as credential theft and remote service exploitation. The simulated attacker in this project uses movement actions consistent with these documented techniques.

2.1.2 Reinforcement Learning

One of the experimental conditions in this project deploys an intelligent defence agent capable of learning and adapting its behaviour during simulation. Reinforcement Learning (RL) was selected as the learning mechanism for this agent. A summary of RL is provided below; more detail on the specific DQN variant used is available in Section 2.2.3.

RL is a method by which an AI agent learns to make decisions through experience. Unlike traditional programming where a developer explicitly writes rules for every situation, an RL agent is given a goal and a scoring mechanism and then discovers through trial and error which actions lead to better outcomes.

The basic RL loop works as follows; at each moment, the agent observes its current situation (called the state). It selects an action. It receives a numerical score (called a reward) reflecting how good or bad that action was. It then observes the new situation resulting from its action. By repeating this process across thousands of trials, the agent gradually learns which actions tend to lead to higher scores.

Key concept: The Q-value

The agent maintains an estimate called a Q-value for every (situation, action) combination. This estimate represents the total future reward the agent expects to receive by taking that action in that situation.

The agent updates these estimates using the Bellman equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where r is the immediate reward, γ discounts future rewards, and the bracketed term is the error between what the agent expected and what actually happened.

Deep Q-Networks (DQN), introduced by Mnih et al. (2015), replace the Q-value lookup table with a neural network. This allows the agent to generalise from past experience to new, unseen situations; a useful capability when the number of possible states is too large to enumerate.

Gymnasium (formerly OpenAI Gym) is a standard Python library that provides a common interface for RL environments. Every environment exposes two key functions:

step() which accepts an action and returns the new state and reward.

reset() which starts a new episode.

2.1.3 Cyber Attack Simulation

Controlled experimentation with real cyber-attacks is impractical. It is not only illegal and dangerous, but difficult to reproduce. Simulation frameworks provide an alternative allowing the key dynamics of network intrusions to be modelled without the risks of testing on real infrastructure.

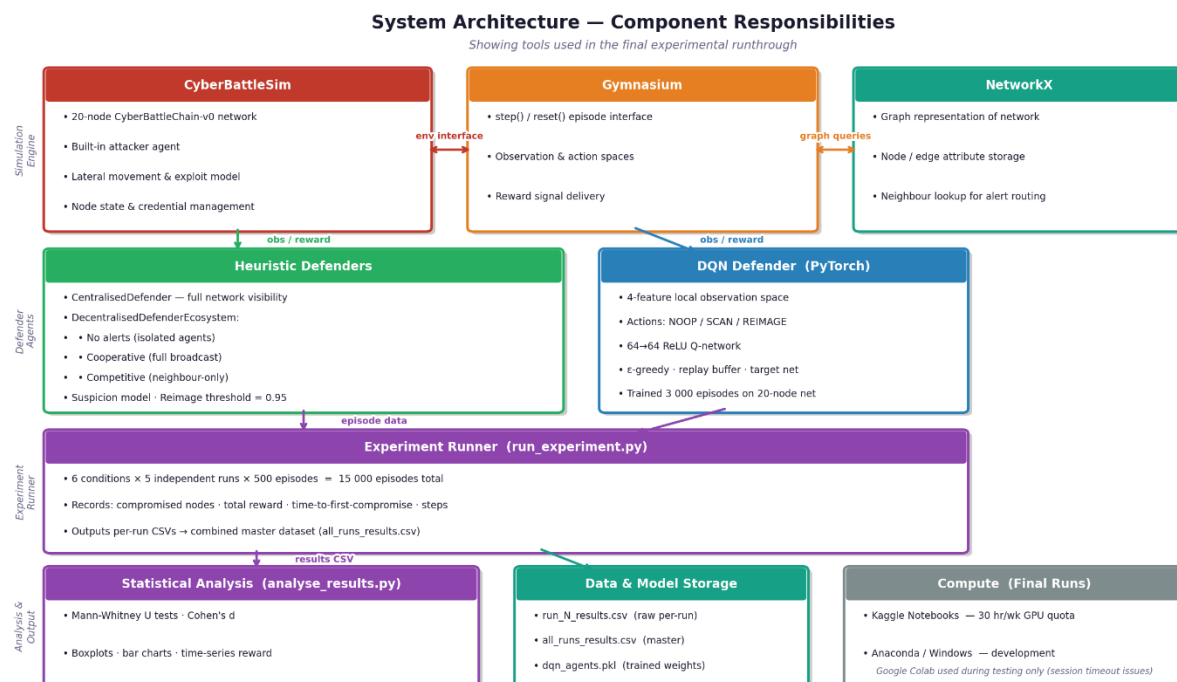


Figure 2. Visual representation of full environment technology stack.

CyberBattleSim (Microsoft Research, 2021) is an open-source simulation framework that models an automated attacker moving through an enterprise network. The attacker selects from a set of valid attack actions each timestep; one or more defender agents may respond. The framework integrates with Gymnasium, making it suitable for RL research.

Related platforms include the CAGE Challenges built on CybORG (Kiely et al., 2023) and CybORG itself (Standen et al., 2021). I chose CyberBattleSim for its Gymnasium compatibility, active maintenance, and modular architecture that allowed the custom multi-agent defender framework required for this project.

Gymnasium was selected as the environment interface because it provides a standardised step-reset loop that is compatible with both CyberBattleSim and the DQN training loop, eliminating custom integration work.

I chose PyTorch for the neural network implementation due to its dynamic computation graph, which simplified debugging the reward hacking failure described in Section 6.7. Python 3.12 was used throughout for consistency with the Kaggle and CyberBattleSim runtime environments.

2.1.4 Multi-Agent Systems

Multi-agent systems distribute decision-making across multiple autonomous agents, each with its own local view of the world. In cybersecurity, this approach has been proposed for intrusion detection (Marchetti et al., 2016) and coordinated network defence (Applebaum et al., 2016). The key advantage over centralised monitoring is resilience. A distributed system has no single point of failure.

The specific frameworks by which multi-agent systems share information and their real-world counterparts are examined in the literature review (Section 2.2.1). The experimental conditions in this thesis are designed to operationalise those mechanisms as testable variables.

2.2 Literature Review

2.2.1 The Changing SOC Paradigm

The structural failures of centralised Security Operation Centre (SOC) teams and the participation barriers of Information Sharing and Analysis Centres (ISACs) establish a clear motivation for automated and distributed defence architectures that do not depend on centralised coordination or voluntary inter-organisation sharing. This thesis directly addresses the ISAC question at an experimental level: Research Question 2 tests whether restricting alert sharing to direct neighbours is defensively equivalent to full network broadcast.

The literature reviewed below suggests there remains important work to do to better understand the optimum level and speed of threat and attack data sharing between organisations.

The current status-quo approach to cyber defence in the majority of organisations is the SOC model. A SOC is a team of highly skilled and specialised talent whose primary goal is to monitor, defend and react to any and all cyber threats across an enterprise network. In recent times as the volume and impact of cyber-attacks has grown, the paradigm has shifted as concerns for the reliability and efficiency of SOC teams is growing.

Islam, C., Babar, M.A., et al. (2025) put forward four root causes to the growing need for alternatives or additions to traditional SOC teams:

1. High security alert volume and velocity (speed of attack).
2. Excessive false positives arising from over-sensitive tuning of Security Information and Event Management (SIEM) tools.
3. An ever-evolving threat landscape challenging even Machine Learning (ML) detection tools.
4. Repetitive triage tasks.

This same paper also cites a Trend Micro survey stating that 51% of SOC teams feel overwhelmed by current alert volume and over 25% of a security analyst's time is spent on false positives. The current SOC model, while standardised, is struggling to scale to both the amount and complexity of emerging cyber threats. This finding is supported by a 2020 study conducted by Vielberth, M., Böhm, F., Fichtinger, I., and Pernul, G. which noted a 65% increase in the number of reported security incidents over the past five years.

A qualitative 2019 study led by Kokulu, F.B, based on eighteen interviews with SOC teams and their managers, revealed major differences in how both groups viewed certain operational and ideological expectations and priorities. The conclusion drawn was that organisational friction is a key factor in the effectiveness of SOC teams, and thus that cyber defence extends beyond just the purely technical aspects.

As the demands on SOC teams increases, the consequences of failure have also increased. The IBM Security and Ponemon Institute (2024) analysed 604 real-world breaches across a range of sixteen countries and discovered the global average cyber breach cost stood at \$4.88M, part of a 10% year-on-year (YoY) increase. Interestingly, organisations suffering from severe staff shortages saw an additional \$1.76M in breach cost, compared to organisations utilising AI and automation who saw a reduction of \$2.2M in breach cost.

With the increases in both volume and potential severity of breaches, many SOC teams feel overwhelmed and unable to keep up with the growing “burnout culture” of cyber security work. A 2021 industry survey conducted by Trend Micro uncovered that 70% of SOC teams feel “emotionally overwhelmed” by the sheer alert volume: with 43% of respondents occasionally turning off alerts and 40% ignoring incoming alerts entirely.

Zooming out a level, from individual organisations to the broader picture at industry and country level, it is important to also consider how organisations collaborate with each other to combat cyber threats. ISACs were put in place as a means of sharing threat information between organisations. ISACs are sector-specific non-profit bodies, such as the Financial Services ISAC (FS-ISAC), through which member organisations voluntarily share cyber threat indicators, vulnerabilities, and incident data with sector peers. According to ENISA (2017), ISACs can be categorised into three main models: country-specific, sector-specific, and international.

While their implementation is a step in the right direction, some key barriers still remain. The MITRE Corporation's 2012 study of the state of ISACs identified the following key issues with the use and adoption of such sharing models:

1. Companies have a natural distrust of their respective governments.
2. Intrusion information that is shared post-attack usually arrives too late to be of use for other organisations.
3. Vulnerability intelligence and reconnaissance is often too general to be immediately actionable by third parties.
4. Anonymisation of organisation and threat data further slows the information flow.

These issues can be seen in a practical sense, as researched in the 2023 study by Chang, K., et al. on “Exploring the Management of Multi-Sectoral Cybersecurity Information-Sharing Networks”. Based on a set of forty in-depth interviews conducted across various sectors within Taiwan, it was found that both formal and informal rules still govern whether ISAC members share or withhold data to each other.

2.2.2 The Case for AI-Driven Defence: Policy Endorsements and Threat Escalation

The first globally agreed guidelines for AI adoption and secure development (which include cyber security guidelines) were only recently formalised in the NCSC (UK) and CISA (US) respectively with the help of twenty-one international partner agencies in 2023. This framework represents the current highest level of government endorsement towards utilising AI within the field of Cyber Security.

To use the UK government as a practical example, it published in 2025 its Code of Practice for the Cyber Security of AI. This voluntary code of practice covers threat modelling for AI-enabled attacks, asset management in the age of AI and disaster recovery. It is currently endorsed by 80% of consultation respondents, serving as a practical example of the UK's growing policy commitments to strengthen AI use in cyber security. Both the 2025 NIST Cybersecurity Framework Profile for AI and the 2022 UK government's National Cyber Strategy (valid through to the year 2030) recognise that AI and ML can both enhance current cyber security practices and amplify cyber risks. This sentiment is echoed in the EU AI Act (fully applicable starting August 2026).

The need for a more comprehensive global cyber security sharing network is growing as the scale and sophistication of cyber-attacks increases. The 2025 Mandiant and Google Cloud Security M-Trends report analysed over 450,000 hours of incident response investigations over the course of 2024. It revealed that 55% of threat groups are financially motivated and that three out of four exploited vulnerabilities were zero-day exploits (exploits where even the vendor is unaware, leading to “zero-days” to fix the issue).

Verizon's 2025 Data Breach Report looked at over 22,000 cyber incidents and over 12,000 confirmed breaches across 139 countries to better understand the cyber-attack landscape. It found that ransomware (malware that encrypts victim data and demands payment for its restoration) was present in 44% of breaches, a 37% YoY increase. It also found third-party cyber-attack involvement doubled to 30% and that the median time between any given vulnerability publication and mass exploitation was zero days.

CrowdStrike's similar 2025 Global Threat Report tracked over 257 known adversaries. It found that the average eCrime breakout time (time taken from initial intrusion to lateral attacker movement through a network) dropped to 48 minutes; with the shortest being 51 seconds.

The 2025 ENISA *Threat Landscape Report* analysed over 4,800 incidents over a period of one year. Over 80% of phishing campaigns used AI-generated or AI-enhanced content. It also uncovered reports of emerging malicious AI systems (e.g. Xanthorox AI) for automating social engineering and malware development.

As a whole these endorsements and threat statistics establish that the case for AI-driven autonomous cyber defence is being made both from ‘policy downward’ and from ‘attacker capability upward.’ CrowdStrike's 51-second breakout time is particularly significant in this regard as it implies human-in-the-loop SOC responses are physically incompatible with modern attack speeds at the fast end of the scale. These figures contextualise the thesis' evaluation environment and justify why autonomous, “always-on” agents are the appropriate and potentially only defensive architecture to investigate.

2.2.3 Deep Q-Networks (DQN)

I chose standard DQN over its extensions (Double DQN, Dueling DQN, Rainbow DQN) because my goal was to demonstrate that learned policy outperforms rule-based heuristics, not to optimise within the DQN family. Those extensions remain appropriate directions for future work once the baseline contribution is established. The body of DQN literature informing this and related design decisions is reviewed below.

Deep Q-Network (DQN) was selected for this project because of three properties that are directly relevant to the cyber defence task: its suitability for discrete action spaces, its established academic use in cybersecurity RL applications, and the interpretability of its Q-values (a property that proved essential for diagnosing the reward hacking failure documented in Section 6.7).

The academic origins of DQNs can be traced back to a 2013 paper published by Mnih, et al. titled "Playing Atari with Deep Reinforcement Learning". An agent partly trained using Q-learning was tasked to learn to play 7 Atari games with only raw pixel values as input. The same publishers later released a follow-up to this experiment in a 2015 paper in which a new agent learnt to play 49 Atari games, again using only raw pixel values as input. This paper also put forward three key innovations in the world of AI and Deep Q-Learning:

1. **Experience Replay** → Storing and random sampling of transitions.
2. **Separate Target Network** → Regularly updated Q-network copy.
3. **Convolutional Neural Network (CNN)-based learning** → Bridges perception and action

These papers served as a catalyst for further study into the Deep-Q Network branch of AI. One year later, a 2016 paper by van Hasselt et al. titled "Deep Reinforcement Learning with Double Q-Learning" uncovered the fact that early implementations of DQNs suffered from substantial Q-value overestimation. The paper investigated and proposed using an adaptation of Double Q-learning to deep networks by using the online network to select actions and the target network to evaluate them.

Another breakthrough paper from 2016 published by Wang, Z. et al. noted that in many of the tested environment states, the action choice barely affects the outcome. Meaning that learning a general state value is more efficient than calculating the effect(s) of a given action on a state. This finding is particularly relevant to cyber-defence where "no attack occurring" states dominate, making the advantage calculations important for distinguishing defensive actions only when they matter. This idea was explored further by Schaul, T., Quan, J., Antonoglou, I., and Silver, D. in their 2016 paper titled "Prioritized Experience Replay". The paper replaced uniform random sampling from the replay buffer with prioritisation proportional to training data error size, ensuring surprising/informative scenarios are replayed more frequently. This finding is critical for cyber defence where reward signals are sparse (attacks are rare events). This study found prioritisation to be the single most important DQN ingredient.

The biggest leap in DQN evaluation came in a 2018 paper published by Hessel et al. They combined six DQN extensions into one agent and evaluated it against the same Atari games previously published. It was found this new "super-agent" had not only better data efficiency but also more stable and accurate final performance. This paper proved to be one of the most influential in the DQN space as it allowed for a more thorough investigation into which enhancements matter most in agent development.

As with all agent development, reward shaping and reward hacking are non-trivial problems that can derail even well-designed models. Amodei, D. et al. published in 2016 their paper titled "Concrete Problems in AI Safety." It presents 5 AI safety problems including reward hacking (where agents exploit unintended loopholes in the reward function). This paper provided a first example into how reward functions can be gamed. It is a foundational reference for understanding why reward design is critical: poorly specified defence rewards could lead agents to exploit the simulation rules rather than learn genuine defence.

Ng, A.Y., Harada, D., and Russell, S.J. published in 1999 "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping." They proposed a potential-based reward function; arguing it is the only function type guaranteeing preservation of the optimal policy (i.e. non-potential-based shaping can change the optimal policy). The penalty adjustment used to resolve reward hacking in this project is consistent with potential-based shaping principles, ensuring the optimal defensive policy is preserved under the modified reward function.

A more concrete example of reward shaping and reward hacking can be found in the 2016 OpenAI blog post by Clark, J. and Amodei, D. titled "Faulty Reward Functions in the Wild." It demonstrates a concrete reward misspecification example: an RL agent placed in a CoastRunners environment (a boat racing, checkpoint-style Flash game from 2011) exploits shaping rewards by circling in the lagoon rather than completing the race. This example has become a widely cited illustration of the risk when reward shaping incentivises "looking busy" rather than effective defence.

The key concepts of specification gaming were put forward in a 2020 DeepMind Blog post by Krakovna, V. et al. titled "Specification Gaming: The Flip Side of AI Ingenuity."¹ It defines it as behaviour satisfying the literal or explicit objective without achieving the intended outcome. This same blog post also catalogues causes (poorly designed shaping, reward tampering, environment exploitation) and provides a comprehensive example list. It lays out the conceptual framework for anticipating how cyber defence agents could game objectives.

Pan, A., Bhatia, K., and Steinhardt, J. also uncovered in 2022 through their paper titled "The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models" that more capable or "intelligent" agents have an increased possibility of finding and exploiting reward functions or agent specifications.

2.2.4 Reinforcement Learning Meets Cyber Defence

The literature outlined below establishes RL-based cyber defence as a maturing research area with both reproducible environments and growing institutional benchmarks. However, prior work consistently relies on centralised training with global state visibility during the training phase. This thesis takes a stricter approach where agents are trained and deployed with local observations only. This is closer to real-world deployment constraints where individual endpoint agents cannot access global network state during either training or execution and represents a gap in the prior literature that this work directly addresses.

The DQN foundations established in the prior section were quickly applied to cybersecurity simulation environments.

¹ Blog post, not peer-reviewed, but extensively cited in academic literature.

One of the first prominent papers delving into this topic was a 2020 paper published by Hammar, K. and Stadler, R. titled "Finding Effective Security Strategies through Reinforcement Learning and Self-Play". The paper proposed modelling cyber-attack and cyber-defence as a Markov game, where the environment changes probabilistically based on the actions taken within it. The emerging policies found by the AI agents were akin to common security knowledge and already began to resemble real human strategies. This paper is credited as one of the first instances demonstrating the viability of RL for network defence.

The following year (2021), Hammar, K. and Stadler, R. published "Learning Intrusion Prevention Policies through Optimal Stopping" in which the RL agents were calibrated using real-world infrastructure data. The simulation-to-emulation methodology demonstrated here informed the decision to train and evaluate agents in CyberBattleSim before any consideration of real network deployment.

Standen et al. later released their paper on "CybORG: A Gym for the Development of Autonomous Cyber Agents". It introduced CybORG (Cyber Operations Research Gym), an OpenAI-based environment for autonomous cyber operations research with both simulation and emulation backends. After this paper's publication, CybORG quickly became the primary environment underlying the CAGE Challenges (one of the most widely used benchmark for autonomous cyber defence RL). The first notable use of CybORG for agent cyber-defence investigations was released in 2023 by Kiely et al. and evaluated RL-based autonomous agents within the CybORG environment. They examined agent performance, effectiveness, resilience, and generalisability in defending enterprise networks against automated red team attacks. This provided both concrete benchmark results and evaluation methodologies directly comparable to this work.

Around the same time as the release of CybORG, the Microsoft Defender Research Team released CyberBattleSim; an experimental research platform from Microsoft for investigating autonomous agent interaction in simulated enterprise networks. It uses high-level abstraction with an OpenAI Gym interface for DQN and other RL algorithms. This marked one of the first instances of industry investment in RL-based cyber security research.

These various research platforms were evaluated in the 2023 paper by Vyas et al. titled "Automated Cyber Defence: A Review". It features a comprehensive survey of automated blue and red team solutions as well as collecting performance results. The paper surveyed all open-source cyber defence gyms (CybORG, YAWNING TITAN, CyberBattleSim, NASim) and evaluated them against a requirements framework.

With the release and subsequent uptick in usage of these various research platforms came the idea of utilising Multi-Agent Reinforcement Learning (MARL) as a tool for cybersecurity. Rashid et al. published in 2018 a proposal for QMIX, a MARL method for Centralised Training with Decentralised Execution (CTDE). The agent's policy is learnt with full network visibility and control but then applied to individual endpoint agents. This CTDE approach maps very well to the architectural approach taken in this thesis.

One of the first papers applying the cooperative MARL method (QMIX and IQL) to cyber-defence was published in 2023 by Wiebe et al. Decentralised agents were trained from scratch on varying network sizes. It offered a first view into the differences between fully decentralised (IQL) vs CTDE (QMIX) architectures. The paper concluded that QMIX can learn effective decentralised defence, directly relevant to this thesis. A follow-up investigation by Singh, A.V., et

al. in 2024 looked into whether splitting the various stages of cyber-defence into subproblems (H-MARL) could offer any measurable benefits. The paper concluded that in large-scale scenarios, the splitting did in fact outperform flat MARL. This alludes to the fact that even centralised agents trained within a global view fail to utilise shared information effectively.

The most recent research into utilising MARL for cyber-defence was published in 2025 by Landolt, C.R., et al. It covers intrusion detection, lateral movement containment, autonomous Intelligent Cyber-defence Agents (AICA), and cyber gyms. This survey provides broader context for the research area and the current state of MARL adoption in industry.

2.2.5 Prior Quantitative Results and Benchmarks

The benchmarks established below highlight compromised nodes, attacker reward, win rate, and episode length as the standard evaluation metrics for autonomous cyber defence. This thesis adopts the primary metrics (compromised nodes and attacker reward) and supplements them with statistical significance testing across five independent runs.

Since the appearance and adoption of AI in cybersecurity, the growing volume of RL-based cyber defence work has produced a set of well-cited studies that serve as de facto benchmarks for the field.

The primary benchmark paper for multi-agent RL in cyber defence was published in 2025 by Kiely, M., Ahiskali, M., Borde, E., Bowman, B., Bowman, D., et al. It explored five defender agents defending seven subnets against automated red team attacks and was measured using cumulative episode reward across four network scenarios. It features between 92 and 242 action spaces, partial observability, false positives/negatives, and variable action durations. It also was responsible for the creation of the MARL leaderboard for autonomous defence. Directly relevant to this work, it also explicitly compares single-agent (centralised) vs multi-agent (decentralised) approaches, noting MARL reduces per-agent state-action space and provides better robustness through distribution.

In terms of agent evaluation, a 2025 study by Morris, B., et al. on "Evaluating Reinforcement Learning Agents for Autonomous Cyber Defence" put forward the Measure of Effectiveness (MoE) metric for evaluating RL agents beyond the standard RL reward measurements. It evaluates agents across a 1,000 episode (each 100 timesteps/days) for general usability and can catch undesired training effects where normal metrics struggle to.

The challenge of reward design in cyber defence RL has received dedicated attention in recent work. Bates et al. (2023) studied the impact of reward signals in CybORG/CAGE 2, finding that cyber rewards are typically penalty-only, and that DRL algorithms are sensitive to penalty magnitude. A follow-up study by Hicks et al. in 2025 challenged the widespread use of "scaffolded" reward functions across cyber gyms, finding that sparse positive reinforcement for uncompromised network states trains more effective and more stable agents across network sizes ranging from 2 to 50 nodes. The reward hacking failure encountered during DQN training (Section 6.7) is consistent with the instability that dense penalty-heavy reward functions produce, and its resolution (increasing the unnecessary reimage penalty to make indiscriminate reimaging unprofitable) aligns with the principle that penalty magnitude is the primary lever for preventing degenerate policies.

A key paper published in 2024 by Hammar, K. and Stadler, R. titled "Causally aware reinforcement learning agents for autonomous cyber defence" looked at using causal

relationships instead of statistical methods for training RL agents. This paper looked at the baseline Key Performance Indicators (KPIs) such as defender/attacker win rate, episode length, and healthy nodes. The causally aware agent win rates exceeded the undefended baselines by ~28% and exceeded the Neural Network (NN) baselines by ~43%.

The value of information sharing between defensive agents has also been studied. Sarhan et al. (2023) compared centralised, localised, and federated intrusion detection approaches, finding that federated learning (agents share model updates rather than raw data) achieved performance comparable to centralised learning while preserving data privacy. Kamhoua et al. (2020) demonstrated that cooperative sharing enables stronger collective defence with less individual investment. These findings set a prior expectation that some form of information sharing improves defensive outcomes. This means that this thesis' result, that restricting sharing to direct neighbours is sufficient, is a meaningful and non-obvious finding.

In the case of evaluating centralised vs decentralised architectures, Kiely et al. (AAAI 2025) put forward the basic fact that, in most cases, “the failure of an agent to defend a single subnet is typically less impactful than the agent failing to defend the entire network.” Thus, making the case for a more decentralised approach. Singh et al. (2024) expanded on this point by stating in their paper “Even though the centralized [agent] was trained jointly, using the global state, it is unable to utilize the shared information effectively”, providing direct evidence that the decentralised MARL architecture outperforms more centralised methods.

2.2.6 Gaps in Literature

The literature reviewed in this chapter reveals four areas that remain unexplored or insufficiently evidenced. These areas are:

1. Prior MARL cyber-defence work uses CTDE architectures (i.e. trained with global state visibility). This thesis tests fully local-observation agents with no shared training.
2. No prior study has isolated cooperative vs competitive alert sharing as an experimental variable (or indeed mapped it directly to the ISAC vs internal SOC distinction).
3. Most benchmarks cited use CybORG with more complex attacker models, whereas CyberBattleSim provides a complementary and more reproducible chain-topology testbed.
4. Reward hacking or specification gaming in cyber defence RL has been documented but rarely resolved and reported as a standalone experimental finding with policy implications.

These four gaps collectively define the contribution space of this thesis. Addressing them together (testing fully local-observation decentralised agents, isolating cooperative versus competitive alert sharing as a controlled experimental variable, using CyberBattleSim's chain topology as complementary to CybORG, and documenting reward hacking as a resolved finding with policy implications) constitutes a coherent and original contribution to the cyber defence literature.

3. Simulation Environment & Experimental Design

This section aims to put forward the environmental conditions of the experiments as well as the methodologies used behind the implementation choices.

3.1 The CyberBattleSim Framework

The key starting point for the setup phase is to select a suitable environment and framework to assess the various research questions posed.

3.1.1 Framework Architecture

CyberBattleSim models a network of machines (nodes) connected by communication links. An automated attacker starts on one machine and attempts to spread to others, accumulating a score proportional to both the number of machines it controls and the time it holds them. This score models real attacker value since a short-lived compromise of a few machines is worth less than sustained control of many.

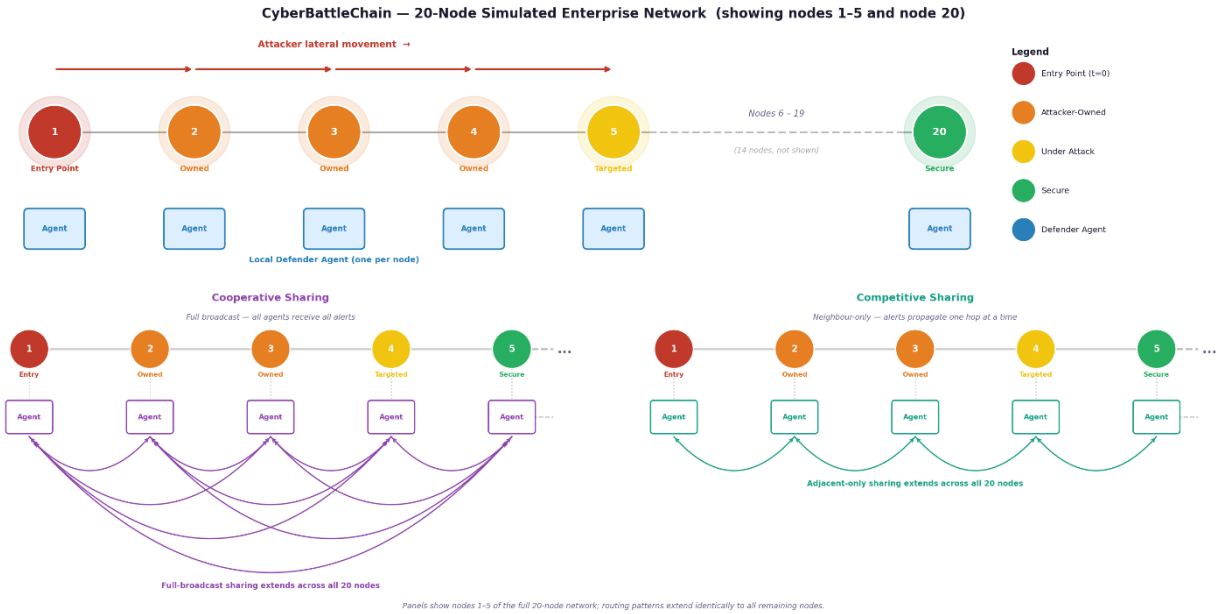


Figure 3. Visual representation of CyberBattleSim network used for this experiment.

Critically, the attacker's score accumulates with time spent on each machine. This means an attacker who gains control of three machines and is removed after 50 steps earns far less than one who holds two machines for the full 200 steps. Defenders therefore benefit not just from evicting attackers, but from evicting them quickly. The primary metric "Compromised Nodes at Episode End" measures containment success and "Attacker Reward" measures both spread and persistence.

Each simulated attack runs for a maximum of 200 timesteps, representing a complete attack lifecycle from initial foothold to either full containment or attacker success. At each timestep, the attacker selects an action; the defender(s) observe their local environment and respond.

3.1.2 Attacker Model

The CyberBattleSim attacker is an autonomous agent that begins each episode on the start node and attempts to spread laterally through the network.

Its action space consists of three categories: local vulnerability exploits (attacks on the current node), remote attacks (using credentials already obtained to compromise adjacent nodes), and connection discovery (probing neighbouring nodes to identify potential attack vectors). At each timestep the attacker selects the highest-value available action from its current node, making it a greedy agent that does not explore; it exploits the best-known option at every step.

The attacker's objective is to maximise cumulative reward, which grows proportionally to the number of machines held and the duration of control. This means the attacker is incentivised both to spread across the network and to resist eviction/reimage. This directly opposes the defender's objective of early containment.

Critically, the attacker follows a pre-determined fixed strategy and does not adapt to defender behaviour. It cannot observe defender actions and does not modify its approach in response to being cleaned off a node. This is a known simplification: real-world attackers adapt, probe defences, and may deliberately slow their spread to avoid detection. The implications of this limitation for generalising the results are discussed in Section 7.4.

3.2 Network Topology

All experiments use a 20-node linear chain network where each machine connects to at most two others, forming a line from node 1 to node 20. The attacker starts at one end and must move one hop at a time.

I chose a linear chain topology because it directly models the lateral movement phase of an attack. Attackers must pass through intermediary machines to reach valuable targets deeper in the network. It also provides a clear test of alert propagation since an alert generated at node 5 takes multiple hops to reach node 18, making the difference between local and network-wide sharing potentially observable.

Initially a linear twelve node design was used. The network was extended to twenty nodes to improve the sensitivity of the cooperative vs. competitive alert sharing comparison.

3.3 Experimental Conditions

Six conditions were evaluated. Each ran for five independent runs of 500 episodes, giving 2,500 episodes of data per condition and 15,000 episodes total.

#	Condition	Description
1	No Defender	The attacker operates entirely unopposed. Establishes the worst-case baseline.
2	Centralised	A single agent with visibility of all twenty machines simultaneously. Analogous to a traditional SOC.
3	Decentralised (no alerts)	One agent per machine, each seeing only its own machine. No communication between agents.
4	Decentralised (cooperative)	One agent per machine. When a machine detects a threat, it broadcasts an alert to every other machine immediately.
5	Decentralised (competitive)	One agent per machine. Alerts shared only with directly connected neighbours — modelling ISAC-style partner sharing.
6	RL Defender (DQN)	One DQN agent per machine using a policy trained for 3,000 episodes on the 20-node network before evaluation. Weights frozen at evaluation. Same architecture as described in Chapter 5. Subject to the same per-agent action constraints as the decentralised heuristic conditions.

Table 2. Six experimental conditions evaluated across five independent runs of five hundred episodes each.

Conditions 1 and 2 establish the baseline for RQ1. Conditions 3 to 5 isolate the sharing variable (RQ2). Conditions 2 vs. 3 test the centralised/decentralised axis (RQ3). Condition 6 addresses RQ4.

3.4 Detection Model: Grounding in Real-World Data

A key design decision is how likely a defender is to notice an attacker in or around its node on any given timestep. Setting this too high makes detection trivially easy; too low makes it impossible. The final value was derived from real-world data.

Mandiant’s M-Trends 2025 report places the global average attacker dwell time at 11 days. Mapping this to a 200-step episode where each step represents one day and each episode represents a complete attack lifecycle gives a mean detection step of eleven. Assuming a consistent level of threat detection, the per-step probability of detection is:

$$\begin{aligned}
 p &= 1 / d_{\text{mean}} \\
 &= 1 / 11 \approx 0.09
 \end{aligned}$$

This means that when an attacker is present on a machine, there is a 9% chance per timestep that the defending agent observes a detection signal. This models an environment where skilled attackers can evade detection for many steps, but not indefinitely.

3.5 Evaluation Metrics

Metric	Description	Importance In Analysis
compromised_nodes	Number of machines with the attacker present at episode end	Primary
total_reward	Total score accumulated by the attacker (lower = better defence)	Secondary
total_attacker_success	Multiplication of compromised_nodes and total_reward	Secondary
time_to_first_compromise	Timesteps until the attacker spreads beyond its starting machine	Supplementary
steps	Timesteps until episode termination	Supplementary
terminated_early	Whether the attacker achieved its objective before the 200-step limit	Supplementary

Table 3. Evaluation metrics collected or calculated per episode. Total Attacker Success is a composite metric capturing both the scale and persistence of a successful attack in a single value.

3.6 Statistical Analysis

All statistical tests were chosen after checking whether the data followed a normal distribution. A Shapiro-Wilk normality test confirmed that all conditions produced non-normal distributions ($p < 0.001$ for all), which is expected given the discrete, bounded nature of the data (the number of compromised machines can only be a whole number between 1 and 20).

The primary significance test is therefore the Mann-Whitney U test, an alternative to the t-test that makes no assumptions about the underlying distribution. Independent samples t-tests are also reported as supplementary results. Practical significance is assessed using Cohen's d , which measures how many standard deviations apart two distributions are (values above 0.2 are considered small, above 0.5 medium, and above 0.8 large). All tests use a significance threshold of $\alpha = 0.05$.

3.7 Implementation Challenges and Solutions

Building a multi-agent simulation on top of an existing framework required navigating several non-trivial implementation challenges. The following documents each issue encountered, the resolution applied, and the rationale for choosing it.

Challenge 1: Node ID Type Mismatch (Silent NOOP Failure)

CyberBattleSim assigns string identifiers to network nodes (for example, 'start_node' and '1_LinuxNode') rather than integers. Initial defender implementations iterated over nodes using Python's `range(node_count)`, producing integer indices that did not match any actual node ID. Every defender action silently failed with no error or warning, and defenders appeared to run correctly while doing nothing for entire episodes.

I fixed this by retrieving node identifiers directly from the environment at runtime using `list(environment.network.nodes)` rather than generating them synthetically. This approach was chosen over a hard-coded mapping because it is robust to any network configuration and requires no assumptions about the simulator's internal naming scheme. The difficulty lay not in the fix itself but in diagnosing the root cause: because CyberBattleSim suppresses action errors rather than raising exceptions, the only observable symptom was uniformly poor defender performance across all early experiments.

Challenge 2: NumPy 2.x Incompatibility

CyberBattleSim was originally written for NumPy 1.x. Using NumPy 2.0 introduced a breaking change to the `np.can_cast()` function that caused a `TypeError` during environment initialisation, where the default Python environment installs the latest NumPy version.

I resolved this by explicitly pinning `numpy<2.0` in the environment setup script. A fork of CyberBattleSim with NumPy 2.x compatibility patches was also investigated but was not stable at the time of writing; pinning to the known-compatible version was more reliable for reproducible experiment runs.

Challenge 3: Agent Reinitialisation Across Episodes (Epsilon Stagnation)

The episode reset function `reset_episode()` was originally implemented to clear all per-episode state, including a flag `_initialized = False`. This caused every local agent to be fully reconstructed at the start of each episode, discarding any learned weights and resetting epsilon to its starting value of 1.0. The symptom was that epsilon appeared stuck near 0.999 regardless of training progress, because it was being reset 500 times per training run rather than decaying continuously across episodes.

I fixed this by separating episode state (suspicion scores, alert inboxes) from agent state (neural network weights, epsilon). Only the former is reset between episodes, the latter persists for the lifetime of a training run. This distinction mirrors the standard Gymnasium training loop where the environment resets, but the agent does not. Identifying the bug required adding explicit epsilon logging after every 100 episodes, which revealed the stagnation pattern immediately.

Challenge 4: Time-to-First-Compromise Metric Always Recording Zero

The `time_to_first_compromise` metric was intended to record the first timestep at which the attacker spread beyond the starting node. The original threshold checked for `compromised_nodes > 0`; however, CyberBattleSim initialises the simulation with the attacker already on the start node, meaning the condition was satisfied at timestep 0 for every

episode without exception. The metric therefore always recorded zero and was useless as a diagnostic.

The threshold was changed to `compromised_nodes > 1`, capturing the moment the attacker reached a second machine. The fallback value when the attacker never spread was also corrected from the final step count to `MAX_STEPS` (200), so that episodes in which defenders successfully contained the attacker to a single node were correctly coded as full containment rather than zero-step containment.

Challenge 5: Compute Platform — Migration from Google Colab to Kaggle

DQN training on Google Colab suffered repeated session timeouts during long runs. Colab disconnects idle sessions after 90 minutes and terminates any session exceeding 12 hours, making it unsuitable for the 3,000-episode training run which required substantially longer uninterrupted execution. Multiple partial training runs were lost to this behaviour before a more suitable platform was identified.

Kaggle Notebooks was selected as the replacement. Kaggle provides 30 hours per week of free compute with no idle timeout: a session remains active for as long as a code cell is executing. The setup is near-identical to Colab, using `/kaggle/working/` for persistent file storage instead of Google Drive. A GPU instance (NVIDIA T4) was initially used but provided negligible speedup since CyberBattleSim's simulation loop is CPU-bound Python. The PyTorch forward-passes through the small 4-64-64-3 network were trivially fast compared to the simulator overhead. All final training and evaluations were therefore conducted on Kaggle's CPU runtime, with checkpointing every 500 episodes to guard against any remaining risk of data loss.

Challenge 6: Reward Hacking — DQN Learning an Always-Reimage Policy

During initial training, Q-value analysis revealed that the REIMAGE action dominated every state, including those with zero detection signal and zero suspicion. The DQN had converged to reimaging every machine at every timestep. Under the original reward function, in which an unnecessary reimage incurred only a -1 penalty, the occasional $+10$ reward from accidentally evicting an attacker made this strategy profitable in expectation. This is an instance of reward hacking and is discussed in detail in Section 6.7 alongside its real-world implications.

I fixed this by increasing the unnecessary reimage penalty from -1 to -5 , making indiscriminate reimaging unprofitable even when occasional positive rewards are factored in. The penalty value of -5 was selected by reasoning about the expected rate of unnecessary reimages on a clean network. The agent would reimage approximately 20 nodes per step for 200 steps, accumulating $-20,000$ in penalties per episode, compared to at most $+200$ from catching attackers. This made the always-reimage policy decisively negative-Expected-Value (EV) and the agent quickly learned state-conditional behaviour on re-training.

4. Rule-Based Defender Architecture

4.1 System Design

All non-RL agents use rule-based (heuristic) defending where each agent follows an explicitly programmed decision logic rather than learned behaviour. These agents serve two purposes: they establish a performance baseline against which the RL agent is compared, and they allow the effect of architectural choices (centralised vs decentralised, sharing vs no sharing) to be isolated from learning effects.

All rule-based defenders share the same underlying decision logic. The difference between conditions lies only in what information each agent can observe and how alerts are distributed.

4.2 Suspicion Accumulation

Defenders do not react to a single detection signal by immediately taking action. Instead, each agent maintains a suspicion score for its own machine, updated at every timestep:

$$s_{t+1} = \text{clip}(s_t + 0.40 \cdot x_t - 0.05 \cdot (1 - x_t), 0, 1)$$

Here x_t is 1 if a detection signal was observed this step and 0 otherwise. Suspicion rises by 0.40 when a signal is observed and decays by 0.05 each step with no signal. The clip function keeps suspicion between 0 and 1. In real-world incident response, analysts gather evidence over time before concluding a threat is real, rather than acting on a single potentially spurious alert.

The decay rate of 0.05 was chosen deliberately to make stale alerts matter less. An alert from 10 steps ago that has not been reinforced will have decayed, meaning defenders respond to ongoing threats rather than historical ones. This decay rate also increases the importance of timely alert propagation between agents, which is what differentiates the cooperative and competitive sharing conditions. In real-world incident response, SOC teams operate at a heightened state of alert immediately during and following a security incident. This alertness reduces over time.

4.3 Decision Policy

Given a current suspicion score, the agent applies two thresholds:

- **Suspicion ≥ 0.50 :** Scan the machine (investigate further, raises suspicion if attacker is present)
- **Suspicion ≥ 0.95 :** Reimage the machine (wipes and restores it, evicting any attacker)

The scan threshold of 0.50 triggers an investigation when enough evidence to warrant a closer look is found, but not enough to take disruptive action. The reimage threshold of 0.95 reflects the high bar organisations set before taking one (or more) machine(s) offline since reimaging disrupts legitimate users and services.

4.4 Alert Sharing Models

The three decentralised conditions differ only in how detection events are communicated between agents. The following code excerpt from `decentralised_defender.py` illustrates the broadcast logic:

```
def _broadcast(self, sender_id, msg):
    if not self.share_alerts:
        return # Condition 3: no sharing
    if self.neighbour_only:
        targets = self._neighbours.get(sender_id, [])
        for defender in self.locals:
            if defender.node_id in targets:
                defender.inbox.append({"from": sender_id, **msg})
    else:
        for defender in self.locals:
            if defender.node_id != sender_id:
                defender.inbox.append({"from": sender_id, **msg})
```

Figure 4. Code excerpt from `decentralised_defender.py` showcasing the broadcast logic. Based on the information sharing rules of each condition, the message is sent to either direct neighbours or all defenders (or none at all).

Condition	Alert Behaviour	Real-World Analogy
No alerts	No communication between agents	Isolated endpoint monitoring
Cooperative	Detection broadcast to all 20 machines immediately	Internal SOC-shared threat platform
Competitive	Detection shared with directly connected neighbours only	ISAC inter-organisation sharing

Table 4. Alert sharing models and their real-world analogues.

4.5 Fairness Constraints

To ensure the comparison between centralised and decentralised conditions reflect architecture rather than action budgets, constraints were applied consistently:

- Centralised defender: may act on all 20 machines per timestep, maximum 2 actions per machine per step.
- Decentralised defenders: each agent limited to its own machine, maximum 2 actions per step.

Without these constraints, the centralised defender could scan every machine every step, a rate that no real security system or team achieves. The action constraints ensure performance differences reflect the architectural difference being evaluated.

5. Reinforcement Learning Defender

5.1 Architecture Overview

The RL defender deploys one trained neural network agent per machine. Each agent observes only its own machine, matching the local-observation constraint of the decentralised heuristic conditions, and selects from the same three actions: do nothing (NOOP), scan, or reimagine.

During training, agents explore freely (trying random actions often) and gradually shift toward actions they have learned produce better outcomes. During the final experiment evaluation, their action weightings are frozen. The RL agents no longer learn but apply what they have learned.

5.2 What Each Agent Observes

Input	Meaning	Range
detected_signal	Was a detection signal observed this step?	{0, 1}
suspicion_score	Accumulated evidence of attacker presence	[0.0, 1.0]
inbox_alerts_norm	How many alerts have been received from neighbour agents (normalised)	[0.0, 1.0]
timestep_norm	Progress through the current episode (normalised)	[0.0, 1.0]

Table 5. The four inputs each DQN agent receives at each timestep.

5.3 Neural Network Architecture

Each agent's decision-making is implemented as a small neural network:

4 inputs → 64 units → 64 units → 3 outputs (one Q-value per action)

I chose this architecture deliberately rather than inheriting a default. Three design choices are worth unpacking.

Why four inputs and not more?

Each agent observes only its own machine by design (as explained in Section 5.2). Expanding the input vector to include data from other nodes would re-introduce the global state visibility that the decentralised architecture is specifically designed to avoid.

The four inputs I chose were:

- detected_signal
- suspicion_score
- inbox_alerts_norm
- timestep_norm

Together, these inputs capture everything a local defender could realistically gather or monitor in a real environment: what it just saw, what it has accumulated, what its neighbours have told it, and how far through the episode it is. I considered adding further signals (e.g. action history,

suspicion of neighbouring nodes) but decided against it. Action history is implicit in `suspicion_score`'s decay, and neighbour suspicion would violate the local-observation constraint.

Why ReLU activation?

I chose ReLU over tanh or sigmoid because it avoids the vanishing-gradient behaviour that slows training on small networks, it's the default in modern DQN implementations (including the original Mnih et al. 2015 Atari network), and introduces no additional hyperparameters to tune. Given the reward-hacking debugging work described in Section 6.7, reducing the number of moving parts in the architecture was a deliberate simplicity choice.

Why two 64-unit hidden layers?

The decision here is based on a capacity-vs-overfitting trade-off. With only four inputs and three outputs, the function the network needs to approximate is low-dimensional; deeper or wider networks would add parameters without adding representational benefit and would risk overfitting. Two 64-unit layers is the standard sizing for small-state-space DQN problems and matches the scale used in comparable cyber-defence RL work (Kiely et al., 2023).

A second copy of the network (the target network) is kept frozen and updated only every 10 episodes. This prevents the agent from chasing a constantly moving target during training, which would cause learning to oscillate or implode.

Past experiences are stored in a replay buffer of 10,000 transitions. At each training step, a random batch of 64 experiences is sampled for learning. This breaks the correlation between consecutive observations that would otherwise make training unstable.

5.4 Reward Function

The reward signal tells the agent how good or bad each action was.

Event	Reward	Rationale
Successful reimage (attacker evicted)	+10.0	Primary positive signal: the defender's core objective
Useful scan (attacker present, signal reinforced)	+1.0	Encourages evidence gathering before decisive action
NOOP on clean machine	+1.0	Modest reward for correct inaction
Unnecessary scan (machine clean)	-0.1	Mild discouragement of uninformative noise
NOOP while attacker is present	-1.0	Penalises ignoring a known threat
Unnecessary reimage (machine clean)	-1.0	Heavy penalty to prevent reward hacking

Table 6a. Original DQN reward function (pre-fix).

However, over the entirety of the training lifecycle, I uncovered that reward hacking was corrupting results. To combat this, I iteratively tweaked the reward function over time to better align with my intended model behaviour.

The final reward function, after resolving the reward hacking issue described in Section 6.7 is shown below in Table 6b.

Event	Reward	Rationale
Successful reimage (attacker evicted)	+10.0	Primary positive signal: the defender's core objective
Useful scan (attacker present, signal reinforced)	+1.0	Encourages evidence gathering before decisive action
NOOP on clean machine	+0.5	Modest reward for correct inaction
Unnecessary scan (machine clean)	-0.1	Mild discouragement of uninformative noise
NOOP while attacker is present	-2.0	Penalises ignoring a known threat
Unnecessary reimage (machine clean)	-5.0	Heavy penalty to prevent reward hacking

Table 6b. Final DQN reward function (post-fix).

5.5 Training

Agents were trained for 3,000 episodes on the 20-node network. Training used epsilon-greedy exploration. At the start, the agent picks a random action with probability $\epsilon = 1.0$ (fully random). This decays by a factor of 0.9990 per episode, reaching $\epsilon = 0.05$ at episode 3,000 meaning the agent acts on its learned policy 95% of the time.

The training curve, shown below in Figure 5, confirms healthy learning. The average reward rose steadily from approximately 16 at episode 100 to 215 at episode 3,000, with epsilon decaying as designed.

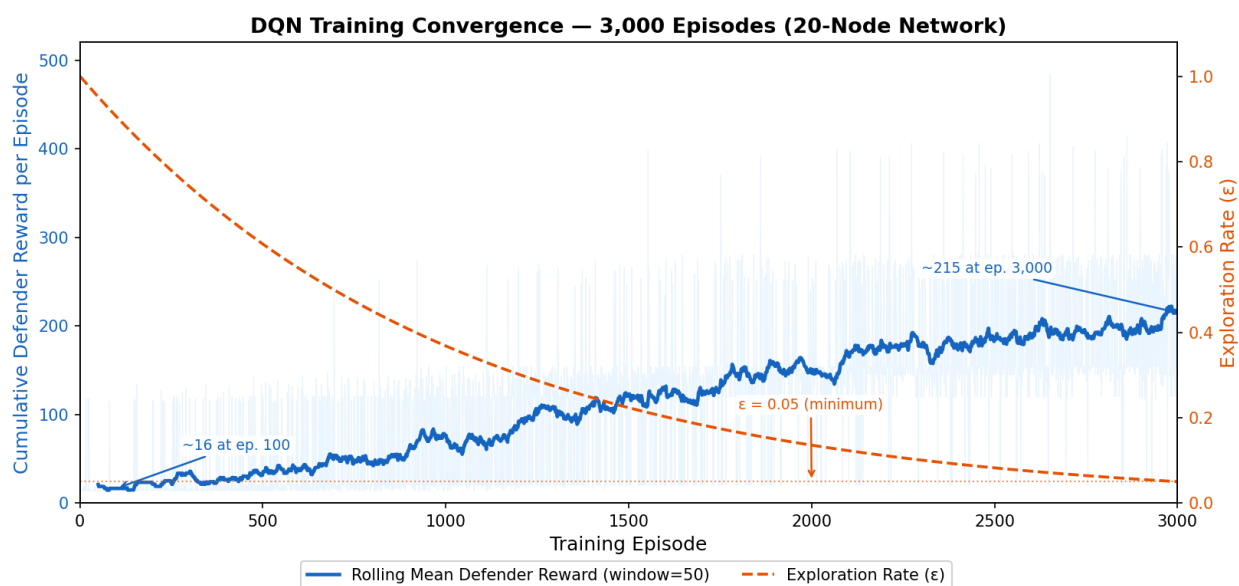


Figure 5. DQN training convergence over 3,000 episodes. The blue line shows the rolling mean cumulative defender reward per episode; raw episode rewards are shown in light blue. The orange dashed line shows the exploration rate (ϵ) decaying from 1.0 to its minimum of 0.05 at episode 3,000. The steady rise in reward alongside smooth epsilon decay confirms healthy learning with no instability.

Training was conducted locally on a standard laptop CPU. The CyberBattleSim simulation loop is CPU-bound Python code, so a GPU provides minimal speedup for this task.

5.6 Hyperparameters

Hyperparameter	Value	Rationale
Learning Rate	0.001	Standard Adam optimiser default; stable for small networks
Discount Factor (γ)	0.95	Moderately future-oriented; balances immediate and long-term reward
Epsilon start (ϵ_0)	1.0	Full exploration at training onset
Epsilon minimum ($\epsilon_{\min}$)	0.05	Retains 5% random exploration at convergence to avoid policy lock-in
Epsilon decay	0.9990	Analytically selected: $0.9990^{3000} \approx 0.050$, reaching minimum at episode 3,000
Batch size	64	Standard for experience replay; balances update stability and compute
Replay buffer size	10,000	Sufficient to store ~50 full episodes; prevents over-sampling recent transitions
Target network update	Every 10 episodes	Stabilises Q-value targets; frequent enough to track learning, infrequent enough to avoid instability
Training episodes	3,000	Required for epsilon to reach minimum on the 20-node network

Table 7. DQN hyperparameters.

6. Results

6.1 Summary of All Conditions

Results are aggregated across all five independent runs, giving 2,500 episodes of data per condition. Standard deviations show episode-level variation within each condition.

Condition	Mean Compromised Nodes	Mean Attacker Reward	Std (Nodes)	Std (Reward)	Total Attacker Success ²
No Defender	2.783	247.40	0.677	81.47	741.4
Centralised	1.850	240.22	0.723	81.38	469.1
Decentralised (no alerts)	1.857	240.84	0.717	82.77	472.4
Decentralised (cooperative)	1.860	238.99	0.733	82.43	470.9
Decentralised (competitive)	1.889	241.45	0.735	81.51	481.7
RL Defender (DQN)	1.510	205.19	0.611	75.61	323.3

Table 8. Summary results across 2,500 episodes per condition (5 runs × 500 episodes).

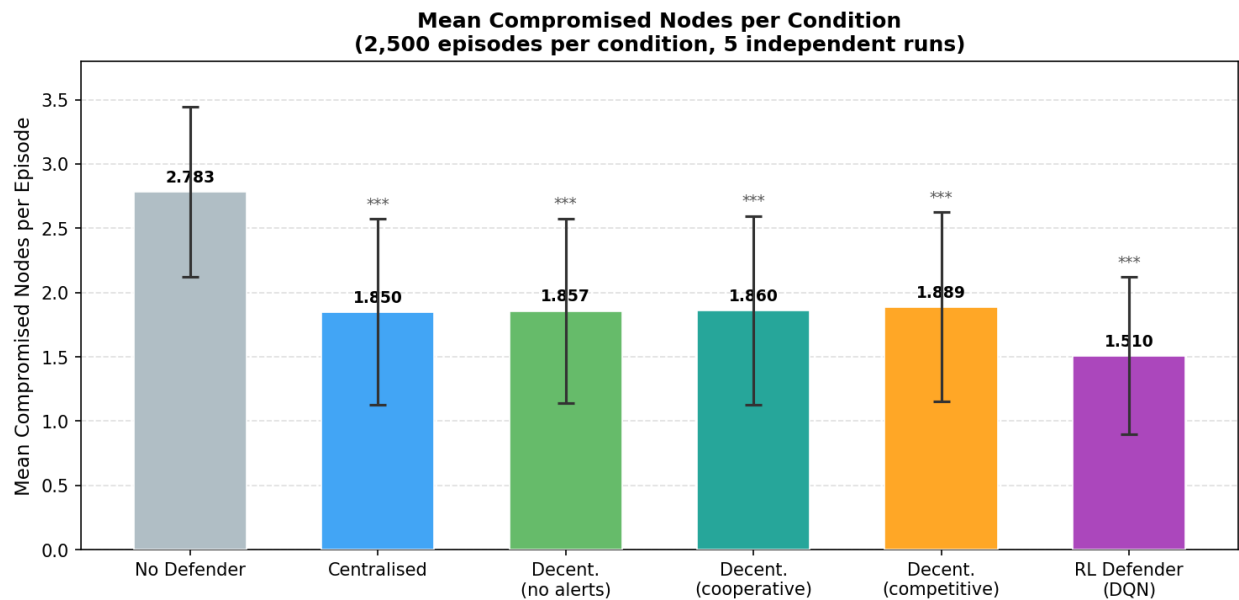


Figure 6. Mean compromised nodes per condition across 2,500 episodes. Error bars show $\pm$ one standard deviation. *** denotes $p < 0.001$ vs the no-defender baseline (Mann-Whitney U test). Lower values indicate better defence.

² Total Attacker Success is calculated as the mean of the per-episode product of compromised nodes and attacker reward, capturing both the breadth and persistence of each attack in a single value.

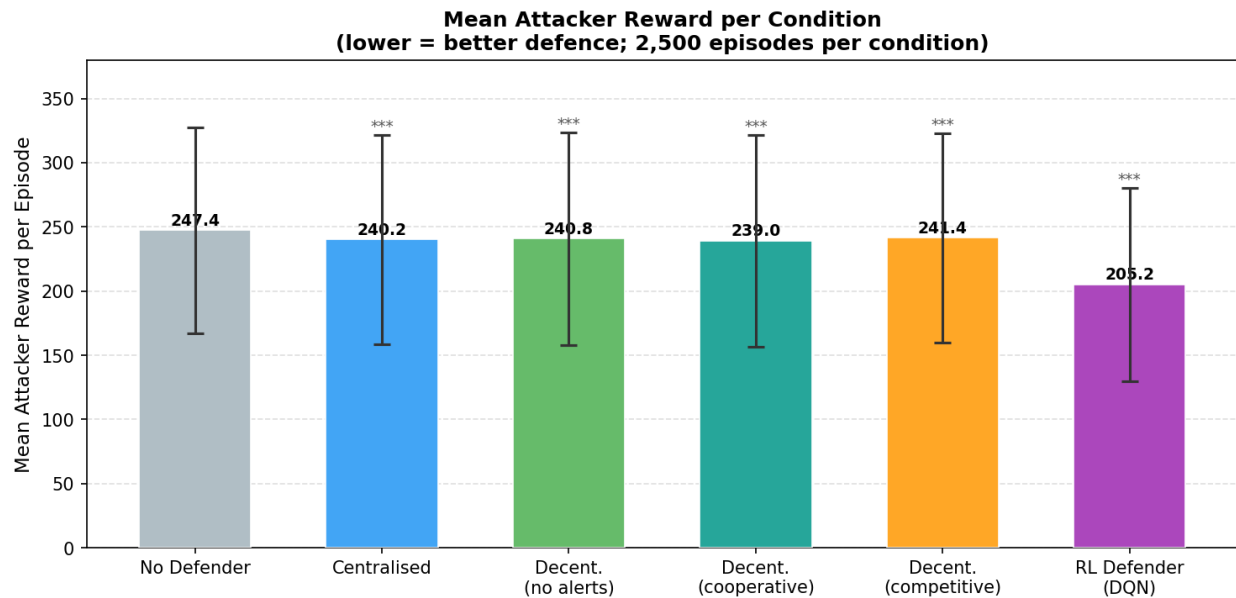


Figure 7. Mean attacker reward per condition across 2,500 episodes. Error bars show $\pm$ one standard deviation. Lower values indicate less value extracted by the attacker. The DQN condition produces a notably larger reduction than all heuristic conditions.

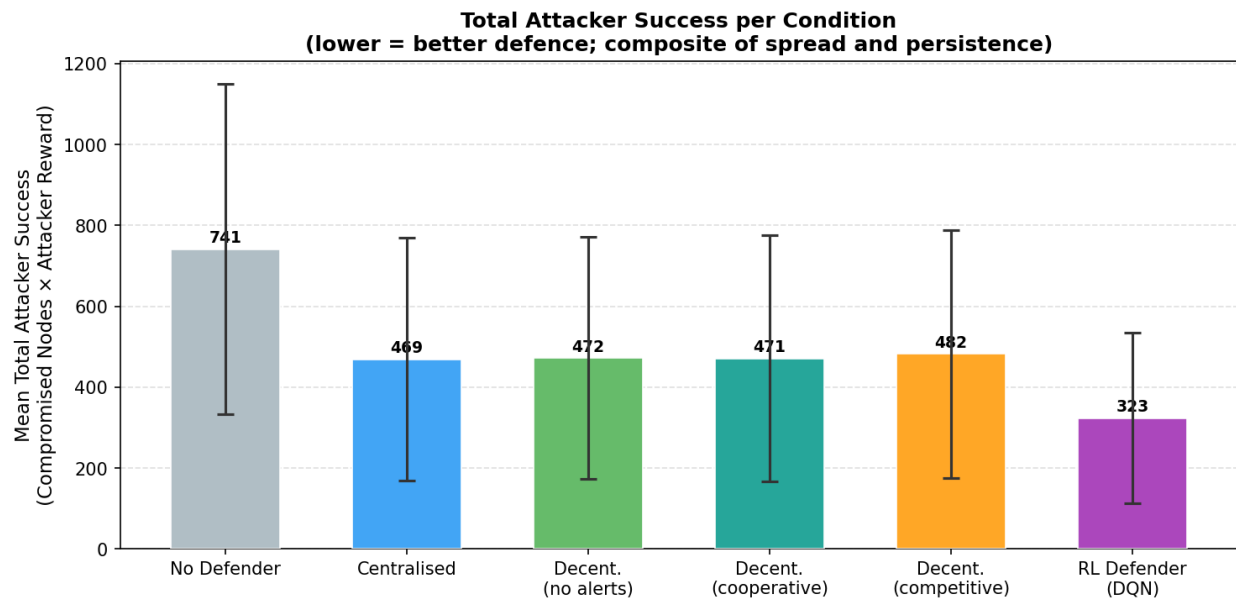


Figure 8. Mean Total Attacker Success per condition.

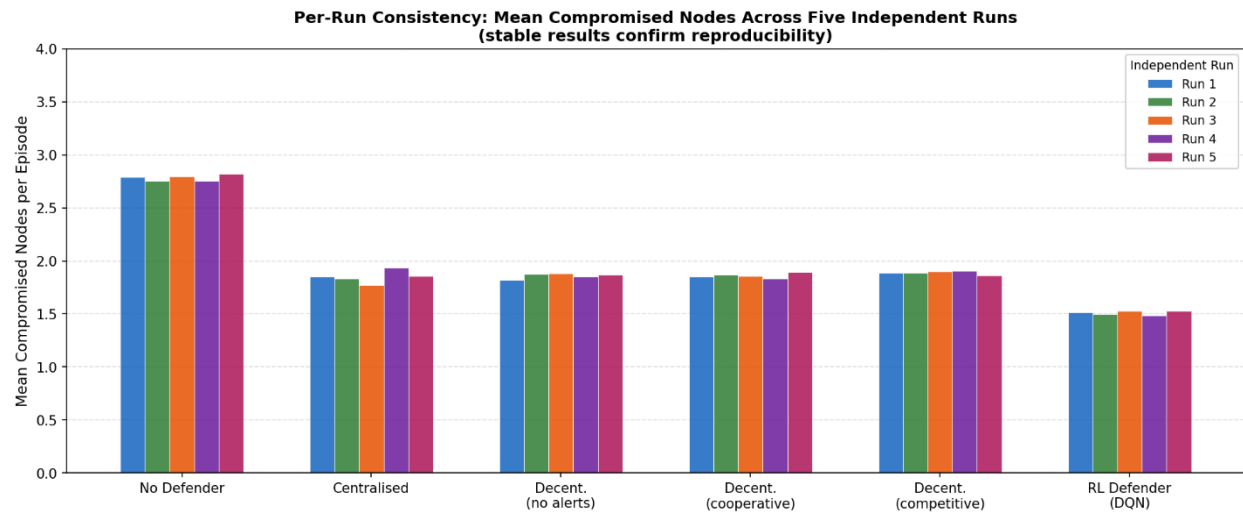


Figure 9. Mean compromised nodes per condition across each of the five independent runs. The near-identical bars across runs confirm result stability.

Two further observations from Table 8 are worth noting. First, the DQN condition (323.3) reduces the combined attack value by 56% relative to no defender (741.4), while all heuristic conditions cluster in the 469-to-482 range; a 35-37% reduction.

6.2 Statistical Significance

Each experimental condition was evaluated across 5 independent runs of 500 episodes, producing distributions of outcomes rather than single values. Because episode-level metrics such as compromised nodes and attacker reward are noisy, observed differences between conditions could arise by chance.

Mann-Whitney (M-W) U tests are used to determine whether the differences between condition pairs are statistically reliable, while Cohen's d quantifies their practical magnitude. Together these tests distinguish genuine performance differences from random variation and protect against drawing strong conclusions from what might simply be noise.

Table 9 reports pairing comparisons between conditions. The p-value indicates whether the observed difference is statistically reliable with asterisks denoting significance levels (***) $p < 0.001$). Cohen's d quantifies practical magnitude independently of sample size, allowing meaningful differences to be distinguished from statistically significant but negligible ones.

Comparison	M-W p (Nodes)	M-W p (Reward)	Cohen's d (Nodes)	Cohen's d (Reward)
Centralised vs No Defender	< 0.001 ***	< 0.001 ***	1.347	0.089
Decent. (no alerts) vs No Defender	< 0.001 ***	< 0.001 ***	1.340	0.080
Decent. (cooperative) vs No Defender	< 0.001 ***	< 0.001 ***	1.321	0.103
Decent. (competitive) vs No Defender	< 0.001 ***	< 0.001 ***	1.278	0.073
RL DQN vs No Defender	< 0.001 ***	< 0.001 ***	1.999	0.543
Cooperative vs Competitive	0.166 (ns)	0.097 (ns)	0.040	0.030
RL DQN vs Centralised	< 0.001 ***	< 0.001 ***	0.509	0.448

Table 9. Statistical comparisons. M-W = Mann-Whitney U (primary test). ns = not significant. Cohen's d: ≥ 0.2 small, ≥ 0.5 medium, ≥ 0.8 large.

Every defender condition reduced both compromised nodes and attacker reward relative to no defender at a highly significant level ($p < 0.001$, Cohen's d on nodes ranging from 1.278 to 1.999), confirming that all defensive strategies provide meaningful protection.

Among the heuristic conditions, the differences are negligible. Cooperative versus competitive sharing yields $p = 0.166$ with $d = 0.040$, indicating that the degree of alert sharing has no statistically meaningful effect on outcomes.

The RL DQN defender stands apart: it significantly outperforms even the centralised heuristic ($p < 0.001$, $d = 0.509$ on nodes, $d = 0.448$ on reward), a medium-to-large effect that demonstrates reinforcement learning can discover genuinely superior defensive policies beyond what rule-based approaches achieve.

6.3 Finding 1 - RQ1: All Defenders Significantly Reduce Compromise

Every defender condition reduced compromised nodes relative to the undefended baseline with overwhelming statistical significance (all $p < 0.001$). The effect sizes on compromised nodes are large across all conditions ($d = 1.278$ to 1.999), confirming that the reduction is not merely a statistical anomaly but represents a genuine and substantial defensive improvement.

In practical terms: without any defence, the attacker compromised an average of 2.78 machines per episode. With any heuristic defender, this fell to approximately 1.85 yielding a 34% reduction. With the DQN defender, it fell to 1.51 resulting in a 46% reduction.

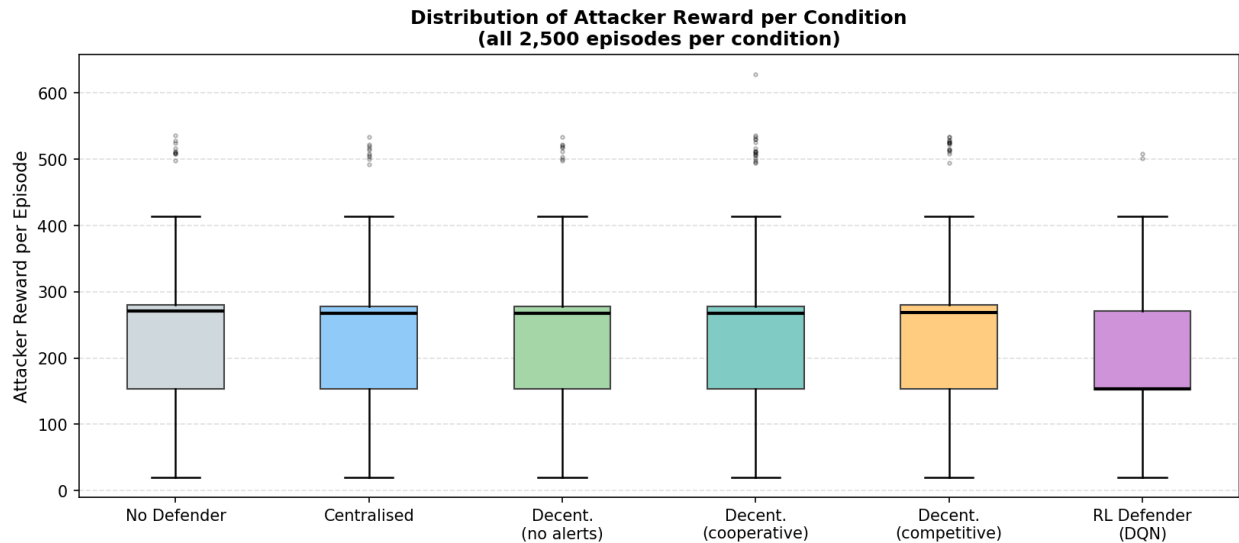


Figure 10. Distribution of attacker reward per condition across all 2,500 episodes. The RL Defender (DQN, rightmost) shows a visibly tighter distribution and lower median than all heuristic conditions. The five heuristic conditions are closely clustered, confirming the null result across sharing configurations.

6.4 Finding 2 - RQ3: Decentralised Agents Match Centralised Performance

The decentralised no-alerts condition (mean 1.857 compromised nodes) is statistically indistinguishable from the centralised defender (mean 1.850). The Mann-Whitney U test between these two conditions yields $p > 0.05$ (not significant), and the difference in means is 0.007 nodes which is less than one hundredth of a machine on average.

This is a significant finding as statistically the two conditions are equivalent. Practically, it means that deploying independent local agents (each seeing only its own machine, with no central coordination) produces the same defensive outcome as a centralised system with full network visibility. The infrastructure overhead, single point of failure, and scaling challenges of centralised monitoring do not translate into better outcomes in this environment.

6.5 Finding 3 - RQ2: Cooperative vs Competitive Alert Sharing

The cooperative (full broadcast) and competitive (neighbour-only) alert sharing conditions produced statistically indistinguishable results on both compromised nodes ($p = 0.166$, Cohen's $d = 0.040$) and attacker reward ($p = 0.097$, Cohen's $d = 0.030$). Effect sizes below $d = 0.05$ indicate a trivially small practical difference.

The underlying reason lies in the chain topology. When the attacker must move one hop at a time, the nodes that most need to receive an alert are precisely the direct neighbours of the detecting node. That is, the machines the attacker will reach next. The competitive condition already delivers alerts exactly there. The cooperative condition's additional reach (alerting machines 15 hops away from the current threat) provides no actionable benefit, because those machines will not face the attacker for many timesteps, and the alert will have decayed by then.

6.6 Finding 4 - RQ4: DQN Outperforms All Heuristics

The DQN defender outperformed all heuristic conditions on both primary metrics with high statistical significance and medium-to-large effect sizes. Against the centralised defender specifically:

Mann-Whitney $U = p < 0.001$,

Cohen's $d = 0.509$ on compromised nodes and $d = 0.448$ on attacker reward.

The DQN's advantage comes from its ability to learn state-aware policies. The heuristic defenders apply the same fixed thresholds regardless of the episode's phase or the attacker's behaviour. The DQN agent, by contrast, can learn to respond differently based on the timestep. For example, being more aggressive late in an episode when the attacker has less time remaining or being more conservative early on when suspicion signals may be noisy.

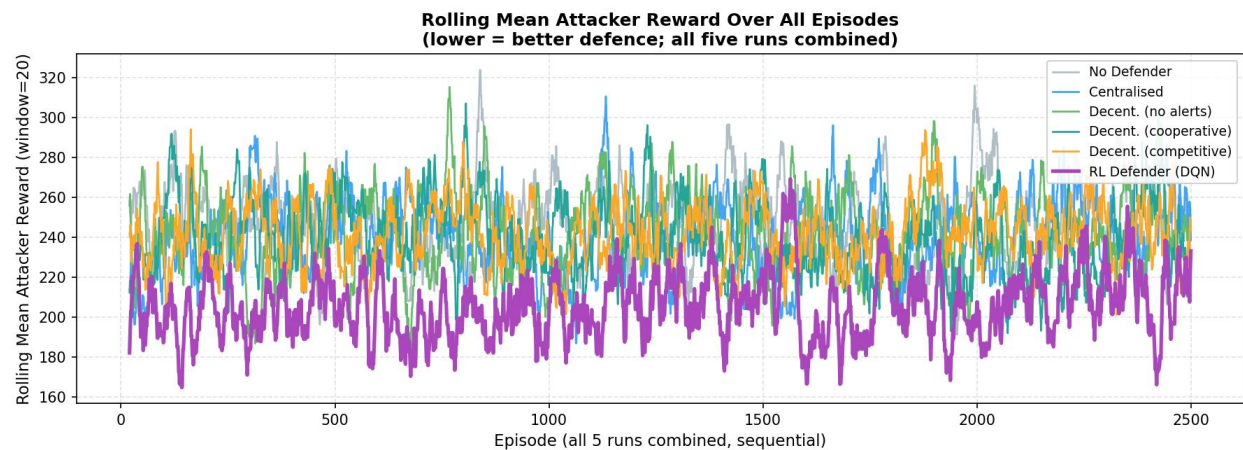


Figure 11. Rolling mean attacker reward over all 2,500 episodes across five combined runs. The RL Defender (DQN, purple) consistently sits below all heuristic conditions throughout training. The five heuristic conditions are tightly clustered, confirming the cooperative vs. competitive null result is stable across episodes.

6.7 Reward Hacking: A Training Failure and Its Resolution

During initial RL training, an unexpected failure was discovered. After training converged, the agent's policy was inspected by examining its Q-values which as discussed previously are the estimates of expected future reward for each action. The analysis revealed that REIMAGE was the highest-valued action in every state, including states with zero detection signal and zero suspicion. States where no rational defender would take disruptive action.

The agent had learned to reimage every machine every step. This maximised its reward under the original reward function because the occasional +10 reward from “accidentally” catching an attacker outweighed the -1 penalty for unnecessary reimaging. The agent was indeed satisfying the reward signal without achieving the intended defensive goal.

What reward hacking looks like in practice

A real-world AI security system that reimages machines constantly would evict any attacker present but would also continuously interrupt legitimate users, consume costly IT resources, and destroy any work in progress. In business operation terms, this is equivalent to a denial-of-service attack against your own infrastructure.

The fix was straightforward once the failure was diagnosed: increasing the unnecessary reimage penalty from -1 to -5 made the always-reimage strategy unprofitable. Even if the agent reimages an infected machine and earns +10, it will reimage far more clean machines over the course of an episode, making the expected reward of indiscriminate reimaging negative. The corrected reward function (Table 6b) produced a policy with a much more appropriate behaviour and better evaluation performance.

This episode illustrates a principle with direct relevance to AI integration and deployment in that reward calibration is a policy decision, not a technical detail. The relative weights determine what the agent actually optimises for, and those weights must reflect real-world constraints including the cost of false positives, not just the benefit of true positives.

7. Discussion

7.1 Decentralised Defence as a Viable Architecture

The most significant finding of this project is that decentralised agents, each operating with only local information and no coordination, match the performance of a centralised defender with full network visibility. This directly challenges the assumption that effective cyber defence requires centralised monitoring infrastructure.

The centralised model has genuine costs that this result calls into question. A SOC requires a central platform capable of ingesting and analysing telemetry from every machine on the network simultaneously. This platform is expensive to build or buy, complex to maintain, and represents a single point of failure. If the SOC loses visibility through attack, overload, or misconfiguration the entire network loses coordinated defence. Decentralised agents have none of these failure modes because each machine defends itself independently, and the loss of any single agent does not degrade the defence of others.

7.2 Implications for Threat Intelligence Sharing Policy (ISAC)

The null result on cooperative vs. competitive sharing is policy relevant. The fear that ISAC-style sharing leaves organisations exposed compared to full company-wide broadcast is not supported by this evidence in chain-propagation network environments. For an organisation evaluating whether to join an ISAC, this finding supports the argument that sharing with direct partners only is defensively sufficient. Broader disclosure of internal threat data to the entire industry does not appear to create additional defensive value in this topology.

Beyond validating current ISAC participation, these findings invite a broader reconsideration of what ISACs could become. If neighbour-only sharing is defensively equivalent to full broadcast during an active incident, the value of ISACs as real-time response networks is limited. While ISACs already have a preventative component (sharing threat intelligence, sector-wide advisories, and emerging indicators of compromise) this role is typically secondary to reactive incident coordination.

These findings suggest that balance could be inverted. Shifting towards early warning, such as monitoring dark web forums for emerging exploit kits, represent a higher-leverage use of shared infrastructure than real-time containment, which local agents appear capable of handling adequately without broad coordination. In this model, the primary value of ISAC membership would lie in compressing the window between a threat's emergence and the industry's awareness of it, moving upstream of the attack rather than in response to it.

This argument carries the same caveat as the core finding: the chain topology constrains the result, and more complex architectures may reveal a genuine advantage for real-time broadcast. This coupled with the fact many organisations face a legal-privilege obstruction to ISAC sharing means that further investigation remains a priority. But based on this evidence, a stronger emphasis on preventative intelligence sharing is a logical next step in the evolution of the ISAC model.

7.3 Implications for Organisations

Three practical lessons for organisations follow from these findings. I discuss each in turn.

Understanding the cost of centralised security

Traditional enterprise security is built around centralised visibility; a model that demands substantial investment in infrastructure, skilled personnel, and continuous operational overhead. A fully staffed Security Operations Centre (SOC) requires not only significant expenditure but sustained expertise across threat intelligence, incident response, and systems administration.

For large or well-established organisations, this cost is often justified by the scale of assets at risk; yet even these organisations are not immune to the fundamental asymmetry that defines modern cyber conflict. An attacker needs only to find and exploit a single vulnerability, while a defender must protect every node, every service, and every user simultaneously. Attackers can operate with low cost (exploit toolkits, automated scanning, freely available malware frameworks...) while defenders bear the ongoing financial and cognitive burden of comprehensive coverage. This asymmetry coupled with the procurement discretion and reputational risk associated with “AI going rogue” means that even well-resourced organisations face a structural disadvantage: the economics of attack consistently favour the adversary.

For smaller enterprises, distributed organisations, and resource-constrained environments, this imbalance is far more pronounced. The centralised model is often simply out of reach, not due to a lack of awareness, but due to the gap between the cost of adequate defence and available budgets. These organisations frequently operate with minimal security tooling, little dedicated expertise, and no realistic path to full SOC deployment, leaving them disproportionately exposed despite facing the same threat landscape as larger peers.

The finding that decentralised local agents achieve statistically equivalent defensive outcomes without requiring centralised coordination or global network visibility suggests a meaningful alternative. By distributing defensive decision-making to the node level, the architecture explored in this project reduces the dependency on expensive centralised infrastructure, potentially lowering both the financial barrier to entry and the operational complexity of maintaining effective security. In turn shifting the economics of defence in a direction that could benefit organisations across the size spectrum.

The reward hacking lesson for AI security procurement

Any organisation evaluating or deploying an AI-driven security tool should understand the reward hacking risk. A tool that earns points for detections may learn to over-detect. A tool that earns points for blocks may learn to over-block. The reward function that governs an AI security agent’s behaviour is equivalent to a policy document in that it defines what the agent actually optimises for, and it must be designed with operational constraints in mind, not just detection performance.

Procurement of AI security tools should include questions about how the agent’s reward function was designed, what penalty is applied for false positives, and whether reward hacking was tested for during development. An agent that has not been evaluated for this failure mode may behave appropriately in testing and catastrophically in production.

To this end, I recommend organisations seeking to delve deeper into the procurement of AI follow some form of “checklist” for AI security tools. In a more practical sense, this could include the following tests and verifications:

- False-positive penalty magnitude verification
- Reward-function disclosure
- Red teaming for specification gaming

7.4 Limitations

Several limitations constrain the generalisation of these findings:

- **Chain topology:** The linear network design constrains the cooperative vs. competitive comparison. Hub-and-spoke, mesh, or hierarchical topologies (which are more common in real enterprise networks) may show different results. More specifically, they may make full broadcast meaningfully better which would overturn the RQ2 result.
- **Simplified attacker model:** The CyberBattleSim attacker follows a fixed strategy and does not adapt to defender behaviour. A sophisticated real-world attacker would observe defender responses and modify its approach accordingly.
- **Single RL algorithm:** Only DQN was evaluated. More advanced algorithms such as PPO or A2C may converge to better policies with fewer training episodes.
- **Simplified network model:** CyberBattleSim abstracts away packet-level behaviour, operating system mismatches, and realistic vulnerability modelling. Real network environments are more complex.
- **Heuristic reward function:** The heuristic defenders’ suspicion parameters were set by hand. Optimisation of these parameters might improve heuristic performance.
- **Human attack surface:** CyberBattleSim assumes a purely technical attacker operating against a purely technical environment; it cannot represent the scenario where an employee clicks a malicious link, reuses a compromised password, or is manipulated into granting access. The majority of successful real-world breaches involve a human element. This social engineering attack surface is entirely absent from the simulation, meaning the defender agents are never tested against one of the most common categories of real-world intrusion.
- **Human-SOC vs AI-decentralised:** A more rigorous comparison would require a decentralised human condition or a centralised AI condition as controls; neither was feasible within the scope of this project, but both are natural next steps.
- **“Neighbour” is ambiguous in real enterprises:** For a financial services firm, a “neighbour” could mean anyone from a competitor, an upstream data provider, a subsidiary, or a service bus. The chain topology’s linear and static neighbourhood model does not hold up to these real-world nuances.

8. Conclusion

This project makes five meaningful contributions to the AI in cyber security space:

- A working multi-agent cyber defence simulation supporting six experimental conditions across five independent runs of 500 episodes each.
- A novel experimental distinction between cooperative (full network broadcast) and competitive (neighbour-only) alert sharing, mapped to real-world intelligence sharing frameworks.
- Empirical evidence that decentralised local-only agents match centralised defender performance (i.e. SOC performance) at statistically equivalent levels.
- Empirical evidence that restricted alert sharing achieves equivalent outcomes to full broadcast, directly relevant to privacy-preserving threat intelligence policy.
- A trained reinforcement learning defender that outperforms all rule-based conditions, with a documented case study of reward hacking and its resolution.

The most real-world policy-relevant finding is the result on alert sharing. Restricting communication to direct neighbours is defensively sufficient in linear networks. Organisations need not choose between privacy and effective collective defence when sharing threat intelligence with sector partners. This has direct implications for ISAC-style frameworks: automated, targeted sharing with directly connected partners appears to capture the defensive benefit of broader disclosure without the associated exposure of sensitive internal threat data to the wider industry. For organisations hesitant to participate in collective defence due to privacy or competitive concerns, this finding offers evidence that limited, structured sharing is sufficient.

The DQN reinforcement learning defender outperformed all rule-based approaches, demonstrating that learned policies can discover more effective defensive behaviours than hard-coded ones. The reward hacking episode provides an instructive case study in the challenges of deploying AI safely in real-world contexts. Reward specification is not a technical detail to be delegated to engineers; it is a policy decision that defines what the system actually optimises for.

Taken together, these findings support the “Internet of AI” defence architecture as a viable and potentially more resilient alternative to centralised monitoring, particularly for organisations where centralised SOC infrastructure is impractical or where single points of failure represent unacceptable risk. Future work should investigate more complex network topologies, adaptive adversaries, and advanced RL algorithms with improved reward shaping.

References

- Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. arXiv:1606.06565.
- Applebaum, A., Miller, D., Strom, B., Korban, C., & Wolf, R. (2016). Intelligent, automated red team emulation. Annual Computer Security Applications Conference (ACSAC).
- Bates, E., Mavroudis, V., & Hicks, C. (2023). Reward shaping for happier autonomous cyber security agents. Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), pp. 221–232.
- Chang, K., et al. (2023). Exploring the management of multi-sectoral cybersecurity information-sharing networks. Government Information Quarterly, 40(4). DOI: 10.1016/j.giq.2023.101870
- Clark, J., & Amodei, D. (2016). Faulty reward functions in the wild. OpenAI Blog. [Blog post, not peer-reviewed]
- CrowdStrike. (2025). 2025 Global Threat Report. CrowdStrike.
- ENISA. (2017). Cooperative models for information sharing and analysis centres (ISACs). European Union Agency for Cybersecurity.
- ENISA. (2025). ENISA Threat Landscape 2025. European Union Agency for Cybersecurity.
- Hammar, K., & Stadler, R. (2021). Learning intrusion prevention policies through optimal stopping. arXiv:2106.07160.
- Hammar, K., & Stadler, R. (2024). Causally aware reinforcement learning agents for autonomous cyber defence. Knowledge-Based Systems, 307, 112521.
- Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M. G., & Silver, D. (2018). Rainbow: Combining improvements in deep reinforcement learning. Proceedings of the 32nd AAAI Conference on Artificial Intelligence, 3215–3222.
- Hicks, C., Mavroudis, V., et al. (2025). Less is more? Rewards in RL for cyber defence. arXiv:2503.03245.
- Hutchins, E. M., Cloppert, M. J., & Amin, R. M. (2011). Intelligence-driven computer network defence informed by analysis of adversary campaigns and intrusion kill chains. Lockheed Martin Corporation. Leading Issues in Information Warfare & Security Research, 1(1), 80.
- IBM Security & Ponemon Institute. (2024). Cost of a data breach report 2024. IBM Corporation.
- Islam, C., Babar, M. A., et al. (2025). Alert fatigue in security operations centres: Research challenges and opportunities. ACM Computing Surveys. DOI: 10.1145/3723158
- Johnson, C., Badger, L., Waltermire, D., Snyder, J., & Skorupka, C. (2016). Guide to cyber threat information sharing. NIST Special Publication 800-150. National Institute of Standards and Technology.

Kamhoua, C., et al. (2020). An improved Shapley value benefit distribution mechanism in cooperative game of cyber threat intelligence sharing. IEEE International Conference on Communications Technology (ICCT 2020).

Kiely, M., et al. (2023). On autonomous agents in a cyber defence environment. arXiv:2309.07388.

Kiely, M., Ahiskali, M., Borde, E., Bowman, B., Bowman, D., et al. (2025). Exploring the efficacy of multi-agent reinforcement learning for autonomous cyber defence: A CAGE Challenge 4 perspective. Proceedings of the AAAI Conference on Artificial Intelligence, 39, 28907–28913.

Kokulu, F. B., Soneji, A., Bao, T., Doupé, A., Zhao, Z., Wang, R., & Ahn, G.-J. (2019). Matched and mismatched SOCs: A qualitative study on security operations center issues. Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19), pp. 1955–1970. DOI: 10.1145/3319535.3354239

Krakovna, V., Uesato, J., Mikulik, V., Rahtz, M., Everitt, T., Kumar, R., Kenton, Z., Leike, J., & Legg, S. (2020). Specification gaming: The flip side of AI ingenuity. DeepMind Blog. [Blog post, not peer-reviewed]

Landolt, C. R., et al. (2025). Multi-agent reinforcement learning in cybersecurity: From fundamentals to applications. arXiv:2505.19837.

Mandiant. (2025). M-Trends 2025. Google Cloud Security.

Marchetti, M., Pierazzi, F., Colajanni, M., & Guido, A. (2016). Analysis of high volumes of network traffic for advanced persistent threat detection. Computer Networks, 109, 127–141.

Microsoft Research. (2021). CyberBattleSim. GitHub.
<https://github.com/microsoft/CyberBattleSim>

MITRE Corporation. (2012). Cyber information-sharing models: An overview. MITRE Technical Report.

MITRE Corporation. (2022). MITRE ATT&CK®: Adversarial Tactics, Techniques, and Common Knowledge. <https://attack.mitre.org>

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). Playing Atari with deep reinforcement learning. arXiv:1312.5602.

Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529–533.

Morris, B., et al. (2025). Evaluating reinforcement learning agents for autonomous cyber defence. Applied AI Letters, 6, e125. DOI: 10.1002/ail2.125

NCSC & CISA. (2023). Guidelines for secure AI system development. National Cyber Security Centre / Cybersecurity and Infrastructure Security Agency.

Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. ICML.

NIST. (2025). NISTIR 8596 (Preliminary Draft): Cybersecurity framework profile for artificial intelligence (Cyber AI Profile). National Institute of Standards and Technology.

Pan, A., Bhatia, K., & Steinhardt, J. (2022). The effects of reward misspecification: Mapping and mitigating misaligned models. *International Conference on Learning Representations (ICLR 2022)*.

Rashid, T., Samvelyan, M., Schroeder de Witt, C., Farquhar, G., Foerster, J., & Whiteson, S. (2018). QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. *Proceedings of the 35th International Conference on Machine Learning (ICML)*, PMLR 80.

Sarhan, M., Layeghy, S., Moustafa, N., & Portmann, M. (2023). Cyber threat intelligence sharing scheme based on federated learning for network intrusion detection. *Journal of Network and Systems Management*, 31, article 3. Springer.

Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). Prioritized experience replay. *International Conference on Learning Representations (ICLR 2016)*. arXiv:1511.05952.

Singh, A. V., et al. (2024). Hierarchical multi-agent reinforcement learning for cyber network defense. arXiv:2410.17351.

Standen, M., Lucas, M., Bowman, D., Richer, T. J., Kim, J., & Marriott, D. (2021). CybORG: A gym for the development of autonomous cyber agents. *IJCAI-21 1st International Workshop on Adaptive Cyber Defense*. arXiv:2108.09118.

Trend Micro. (2021). 70% of SOC teams emotionally overwhelmed by security alert volume. Trend Micro Press Release, 25 May 2021.

UK Government. (2022). National Cyber Strategy 2022. HM Government.
UK Government, Department for Science, Innovation and Technology. (2025). Code of practice for the cyber security of AI. GOV.UK.

van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), 2094–2100. DOI: 10.1609/aaai.v30i1.10295

Verizon. (2025). 2025 Data breach investigations report (18th ed.). Verizon Business.

Vielberth, M., Böhm, F., Fichtinger, I., & Pernul, G. (2020). Security operations center: A systematic study and open challenges. *IEEE Access*, 8, 227756–227779. DOI: 10.1109/ACCESS.2020.3045514

Vyas, S., Hannay, J., Bolton, A., & Burnap, P. (2023). Automated cyber defence: A review. arXiv:2303.04926.

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., & de Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, PMLR 48, 1995–2003.

Wiebe, J., Al Mallah, R., & Li, L. (2023). Learning cyber defence tactics from scratch with multi-agent reinforcement learning. arXiv:2310.05939.

Appendix A: Project File Structure

File	Purpose
train_dqn.py	Trains the DQN defender for 3,000 episodes. Saves checkpoints every 500 episodes. Resumes automatically from the latest checkpoint if the process is interrupted.
run_experiment.py	Runs all 6 conditions across 5 independent runs of 500 episodes each. Writes per-run CSVs and a combined master dataset.
analyse_results.py	Reads the master CSV and produces Mann-Whitney U tests, Cohen's d effect sizes, and four plots.
dqn_defender.py	DQN agent and defender ecosystem. Defines the neural network architecture, replay buffer, training loop, and reward function.
Centralised_defender.py	Rule-based centralised defender with global visibility.
decentralised_defender.py	Rule-based decentralised ecosystem supporting all three alert sharing modes.
results/dqn_agents.pkl	Trained DQN agent weights. Generated by train_dqn.py. Required by run_experiment.py.
results/all_runs_results.csv	Combined master dataset: 15,000 rows (2,500 per condition).

Appendix B: Development Environment

Component	Details
Simulation framework	CyberBattleSim (Microsoft Research, 2021)
Python version	3.12
Key dependencies	gymnasium==0.29.1, numpy==1.26.4, torch, scipy, matplotlib
Training hardware	Kaggle CPU. GPU not required - CyberBattleSim is CPU-bound.
Random seed	Not fixed. Results are averages across 5 independent runs.
Total episodes	15,000 evaluation episodes (2,500 per condition) + 3,000 DQN training episodes

Appendix C: Original vs. Final Project Plan

Project Evolution

The project evolved from the initial brief. The table below documents the key parameters and decisions as originally planned and as finally implemented, along with the rationale for each change.

Parameter / Decision	Original Plan	Final Implementation	Rationale for Change
Network size	12 nodes	20 nodes	On a 12-node chain, all nodes are within a small number of hops of each other, limiting the sensitivity of the cooperative vs. competitive alert sharing comparison. 20 nodes increase the maximum propagation distance to better differentiate the two sharing models.
Detection probability	$p = 0.15$ (100–200 day dwell time)	$p = 0.09$ (11-day dwell time)	Recalibrated to the 11-day mean attacker dwell time reported in Mandiant M-Trends 2025.
Suspicion increment / decay	+0.30 / –0.02	+0.40 / –0.05	Faster decay makes stale alerts matter less and increases the consequence of timely alert propagation, improving the sensitivity of the cooperative vs. competitive comparison.
Number of independent runs	1 run × 500 episodes	5 runs × 500 episodes	Multiple runs reduce the influence of random seed variance, tighten confidence intervals, and allow inter-run consistency checks.
DQN training episodes	~1,500 episodes (12-node target)	3,000 episodes (20-node network)	The larger 20-node network requires more agent-environment

Parameter / Decision	Original Plan	Final Implementation	Rationale for Change
			interactions for epsilon to decay to its minimum. Epsilon decay was adjusted to 0.9990 to reach $\epsilon = 0.05$ exactly at episode 3,000.
Epsilon decay rate	0.9985	0.9990	Recalculated analytically to reach $\epsilon=0.05$ after exactly 3,000 episodes: $0.9990^{3000} \approx 0.050$. The original value of 0.9985 would reach this threshold too quickly for the extended training run.
DQN unnecessary reimage penalty	-1.0	-5.0	The original penalty was insufficient to deter reward hacking. At -1.0, indiscriminate reimaging was profitable. Increasing to -5.0 made the always-reimage policy decisively unprofitable and resolved the failure mode.
Compute platform	Google Colab (T4 GPU)	Kaggle Notebooks (CPU)	Colab session timeouts caused repeated data loss during long training runs. Kaggle provides 30 hours/week of free compute with no idle timeout. GPU offered no speedup as the bottleneck is CPU-bound simulation, not neural network forward passes.
Competitive alert sharing	Not in original scope	Added as 5th experimental condition	Literature review identified ISAC-style selective sharing as the dominant real-world model.

Parameter / Decision	Original Plan	Final Implementation	Rationale for Change
			Adding a neighbour-only condition enabled direct testing of whether full broadcast vs. restricted sharing has a meaningful defensive cost.

Table C1. Key design decisions showing original plan versus final implementation.

Project Timeline

The project was structured across four phases. The original plan assumed a linear progression through each project phase. In practice, the transition between phases was less clean. Platform issues and reward hacking required revisiting earlier phases during the RL training period. The table below shows the intended and actual timeline.

Phase	Originally Planned	Actual	Notes
Phase 1–2: Environment Setup & Validation	Oct – Nov 2025	Oct – Nov 2025	On schedule. Node ID bug identified here.
Phase 3: Heuristic Defender Architecture	Nov – Dec 2025	Nov 2025 – Jan 2026	Extended due to parameter recalibration (detection probability, suspicion decay) and addition of competitive sharing condition.
Phase 4: RL Defender Design & Training	Jan – Feb 2026	Jan – Mar 2026	Extended due to reward hacking diagnosis, reward function redesign, Colab → Kaggle migration, and network scale-up to 20 nodes.
Final Experiments & Write-up	Feb – Mar 2026	Mar – Apr 2026	Final 5-run experiments completed on Kaggle. Statistical analysis and thesis write-up

Phase	Originally Planned	Actual	Notes
			completed on schedule.

Table C2. Project timeline: original plan versus actual delivery.